

How Hard Can It Be to Formalize a Proof?

Lessons from Formalizing CryptoBox Three Times in EasyCrypt

François Dupressoir¹, Andreas Hülsing^{2,3}, Cameron Low¹, Matthias Meijers²,
Charlotte Mylog⁴, and Sabine Oechsner⁵

¹ University of Bristol

² Eindhoven University of Technology

³ SandboxAQ

⁴ Norwegian University of Science and Technology (NTNU)

⁵ Vrije Universiteit Amsterdam

`authors.proof3ways@proton.me`

Abstract. Provable security is a cornerstone of modern cryptography, aiming to provide formal and precise security guarantees. However, for various reasons, security proofs are not always properly verified, possibly leading to unwarranted security claims and, in the worst case, deployment of insecure constructions. To further enhance trust and assurance, machine-checked cryptography makes these proofs more formal and rigorous. Unfortunately, the complexity of writing machine-verifiable proofs remains prohibitively high in many interesting use-cases. In this paper, we investigate the sources of this complexity, specifically examining how the style of security definitions influences the difficulty of constructing machine-verifiable proofs in the context of game-playing security. Concretely, we present a new security proof for the generic construction of a PKAE scheme from a NIKÉ and AE scheme, written in a code-based, game-playing style à la Bellare and Rogaway, and compare it to the same proof written in the style of state-separating proofs, a methodology for developing modular game-playing security proofs. Additionally, we explore a third “blended” style designed to avoid *anticipated* difficulties with the formalization. Our findings suggest that the choice of definition style impacts proof complexity—including, we argue, in detailed pen-and-paper proofs—with trade-offs depending on the proof writer’s goals.

1 Introduction

The ability to reason about (in)security is a central tenet of modern cryptography. In recent years, machine-checked cryptography [7] has aimed to make this

F.D. is funded in part by Innovate UK SCHEME (Safety Critical Harsh Environment Micro-processing Evolution) project (10065634). A.H. and M.M. are supported by an NWO VIDI grant (Project No. VI.Vidi.193.066). C.L. is fully funded by EPSRC Doctoral Training Partnership 2765784.

reasoning more formal and rigorous, increasing assurance and trust in cryptographic standards and constructions. Yet, these improved guarantees undeniably come at the cost of increased upfront effort in producing a proof. Once a proof exists, though, it is relatively easy for a proofreader to assess the validity (but not the adequacy) of security claims. Since a result is typically proved only once but may be verified many times, this trade-off would normally be reasonable. However, the current cost of producing machine-verifiable cryptographic proofs limits their application to only the most sensitive results. In this work, we seek to explore the sources of this cost, focusing on those that stem from the (pen-and-paper) definition style and so are *not* tool-specific. Specifically, we qualitatively investigate the following hypothesis:

The style in which security notions are expressed affects the cost of formalizing the corresponding proofs.

Although we are motivated by a desire to formalize cryptographic proofs, our findings also offer a way to simplify checking pen-and-paper proofs through basic adjustments to security definitions. Importantly, we see this work as a necessary preliminary study in identifying the key questions that must be addressed to build scalable tools. As such, beyond specific challenges arising with a certain combination of proof, method, and tool, we are interested in issues revealing inherent difficulties that are often glossed over on paper but cannot be ignored in a more formal setting, regardless of the foundation. These issues must be identified and understood before tools can make addressing them more intuitive.

Marking the beginning of a larger investigation into the general challenges outlined above, this work focuses on a specific class of security notions—code-based, game-playing security notions—which underlie the majority of machine-checked cryptographic proofs in the literature. To test our hypothesis, we develop multiple versions of the same security proof in the same tool, independently written by different proof writers in varying “proof styles.” While this approach does not allow for definitive, general conclusions about the cost of formalizing cryptographic proofs, the qualitative analysis provides insights that can be generalized later. (Particularly, given the small number of people proficient in both cryptography and the relevant tools, a multi-tool study must rely on micro-benchmarks. This work can inform the design and selection of such benchmarks.) Specifically, we consider (1) the proof that the generic composition of a Non-Interactive Key Exchange (NIKE) and an Authenticated Encryption (AE) scheme, a well-known instantiation of which is NaCl’s **CryptoBox**,⁶ is secure as a Public-Key Authenticated Encryption (PKAE) scheme;⁷ (2) a traditional code-based, game-playing style à la Bellare and Rogaway [11], the State-Separating Proofs (SSP) style of Brzuska, Delignat-Lavaud, Fournet, Kohbrok and Kohlweiss [17], and a “custom” intermediate, or *blended*, style designed to avoid the difficulties we anticipate from the other two styles; and (3) the interactive theorem prover EASYCRYPT.

⁶ <https://nacl.cr.yp.to/box.html>

⁷ For brevity, we simply refer to the generic NIKE+AE composition as **CryptoBox**.

Keygen()	Enc(k, n, m)	Dec(k, n, c)
$k \leftarrow \mathcal{K}$	$x \leftarrow \text{PRF}(k, n)$	$x \leftarrow \text{PRF}(k, n)$
return k	return $x \oplus m$	return $x \oplus c$

Fig. 1. Example: Nonce-based encryption scheme using a PRF.

In the remainder of this section, we provide an overview of the methodology and discuss our findings. Then, in Section 2, we formally define our objects of study, including the relevant definitions and a high-level explanation of each proof. Afterward, we present and discuss the proofs in Sections 3, 4, and 5. Section 5 also explores, more locally, the precise effect of some fine-grained changes to the definition and proof style, informed by observations made in the three independent proofs. Finally, we conclude in Section 6.

1.1 Methodology

The key components of our methodology include the (selection of) proof styles, tools, proofs, proof authors, and the overall approach. We elaborate on each of these aspects below.

Choice of Proof Styles. As alluded to before, the styles we consider can all be categorized as code-based and game-playing. Loosely speaking, this means that definitions are well-defined probabilistic programs (code-based), including schemes and security notions. If a definition is defined with respect to some (abstract) adversarial algorithm, it is called a game; then, proofs proceed by incremental adjustments to the game(s), justifying each and eventually combining them to obtain the desired result (game-playing). In this work, we compare the *de facto* code-based, game-playing style (with some variation) as proposed by Bellare and Rogaway [11]; a more recent contender in the SSP style as proposed by Brzuska et al [17]; and a “blended” style we devised for this study in an attempt to anticipate and circumvent the difficulties of the other styles. Before further detailing our methodology, we provide a brief primer on these three styles through a simple example of a proof for a nonce-based encryption scheme using a PseudoRandom Function family (PRF). For brevity, we omit some of the formal definitions in this example, but a high-level understanding should suffice.

Example. Consider a PRF $\text{PRF} : \mathcal{K} \times \{0, 1\}^a \rightarrow \{0, 1\}^b$ for some key space $\mathcal{K}$ and $a, b \in \mathbb{N}_{>0}$. The input and output space respectively coincide with the nonce and plaintext space of the encryption scheme. Using this PRF, the encryption scheme shown in Fig. 1 is secure against chosen-plaintext attacks with nonce-respecting adversaries. (Here, $\oplus$ stands for the usual exclusive-or.) Intuitively, security follows directly from the way the PRF is used and the fact we consider nonce-respecting adversaries: the PRF-key is sampled uniformly at random and each input (nonce) is provided only once. By the PRF assumption, each computed mask x is therefore indistinguishable from a uniformly random one, and

effectively functions as a one-time pad for the encryption. In the context of code-based, game-playing security proofs, one might define the security of the encryption scheme using a distinguishing game, similar to IND\$-CPA security, and then show indistinguishability via a reduction from the PRF assumption. The proof styles we consider vary in how this process is carried out, i.e., how security is defined and established.

Bellare-Rogaway Style. Borrowing notation from Bellare and Rogaway [11], we denote by $\text{Adv}(\mathcal{A}^G, \mathcal{A}^H)$ the distinguishing advantage of adversary $\mathcal{A}$ for games G and H . Let $\mathcal{A}$ be an adversary against the security of the construction above. Then, in a security proof, we want to bound the distinguishing advantage $\text{Adv}(\mathcal{A}^{\text{Real}}, \mathcal{A}^{\text{Ideal}})$ between a real game Real and an ideal game Ideal. In the style of Bellare and Rogaway [11], such a proof would use a number of intermediate games and show indistinguishability between each consecutive game, as follows. (We omit the relatively standard game definitions from the description.)

Game Real: Oracle encrypting as in Fig. 1. Here, the encryption key is generated by the game (once) using KeyGen and fixed throughout.

Game G_1 : Replace PRF by a random function F . This replacement is justified by a reduction from the PRF assumption. First, the PRF is “externalized” as a separate oracle in the real game, including both key generation *and* function evaluation using the generated key. Then, the PRF assumption can be applied. Doing this constructively shows the existence of a reduction $\mathcal{B}$ against the PRF security of PRF, as captured by the distinguishing games PRF^0 (“Real PRF”) and PRF^1 (“Ideal PRF”). That is,

$$\text{Adv}(\mathcal{A}^{\text{Real}}, \mathcal{A}^{G_1}) \leq \text{Adv}(\mathcal{B}^{\text{PRF}^0}, \mathcal{B}^{\text{PRF}^1}).$$

Game G_2 : Replace F by uniform sampling. This replacement is justified by the fact that, since we consider a nonce-respecting adversary, the encryption oracle computes F on a fresh domain element each time it is called. Hence, the two games are perfectly indistinguishable, i.e., $\text{Adv}(\mathcal{A}^{G_1}, \mathcal{A}^{G_2}) = 0$.

Game Ideal: Oracle “encrypting” by uniformly sampling ciphertexts.

The transition from G_2 to Ideal follows directly from the (multi-instance) security of the one-time pad—the perfect indistinguishability of their use from uniform sampling, provided they are independent and uniform—captured by the games OTP^0 and OTP^1 . Now, an approach analogous to that in the PRF-based step leads to a reduction $\mathcal{B}$ for which

$$\text{Adv}(\mathcal{A}^{G_2}, \mathcal{A}^{\text{Ideal}}) \leq \text{Adv}(\mathcal{B}^{\text{OTP}^0}, \mathcal{B}^{\text{OTP}^1}) = 0.$$

By transitivity of the advantage function, we can conclude that for our adversary $\mathcal{A}$, there exists a PRF adversary $\mathcal{B}$ such that

$$\text{Adv}(\mathcal{A}^{\text{Real}}, \mathcal{A}^{\text{Ideal}}) \leq \text{Adv}(\mathcal{B}^{\text{PRF}^0}, \mathcal{B}^{\text{PRF}^1}).$$

In a practical proof of the above, the first two steps might be merged into a single one. Note that, in general, proof steps may require repeated externalization of (potentially the same) oracles.

Although reductions that rely on externalizing oracles corresponding to an assumption are generally simple on paper, even the simplest of these may require considerable effort to formalize. For example, consider the real game and the reduction with an explicit PRF oracle, constructed during the first game transition above. When formalizing a complete proof for such a step, it is necessary to formally relate the state distribution of the real game to that of the reduction with the PRF oracle. These relations are expressed as *program invariants*; in this case, the PRF key stored in the *real game* has to follow the same distribution as the PRF key stored in the *PRF oracle*. Then, formally, it must be shown that none of the adversary’s interactions with the oracle can interfere with these invariants. Such proofs may either be fully automated, require the user to identify and provide the invariant (but then would automatically check that the invariant holds), or require the user to both provide the invariant and the proof that it is preserved under adversary interactions. Equivalences that involve moving individual variables from one experiment to another can typically be proved automatically. However, invariants involving functional correctness—such as specifying the precise value of a variable as a function of others, or relating two sampling operations that occur at different points in the game—usually cannot be fully automated. These types of proofs often require the proof writer to provide detailed reasoning, which the tool then checks.

State-Separating Proof Style. State-separating proofs [17] were proposed with the explicit goal of supporting formal verification, as incarnated in the F*-based formal verification methodologies that led to the development of miTLS [13]. The original abstract claims that the method “facilitates computer-aided proofs, inasmuch as the perfect reductions steps can be automatically discharged by proof assistants.” [17, abstract]. The SSP style has been used to develop a number of security proofs on paper, including MLS [15], TLS [16] and garbling schemes [18]. In the context of EASYCRYPT, Dupressoir, Kohbrok and Oechsner [22] have studied the use of SSP as an alternative to the traditional game-playing style.

The SSP methodology rests on two key observations: first, avoiding repeated externalization steps helps reduce the tedium of formalizing them; and second, modularity enables more localized reasoning, thereby keeping invariants simple(r). The starting point is still the game-playing style, but now the game G itself is treated like an adversary and allowed to interact with other games. In the notation of Bellare and Rogaway, we can express an adversary $\mathcal{A}$ interacting with such a game as $\mathcal{A}^{(G^H)}$. The same expression can be interpreted as $(\mathcal{A}^G)^H$, another adversary $\mathcal{A}^G$ interacting with game H , just by moving the boundaries of the adversary. So, if we could express the game we are truly interested in—say, the real security game *Real*—in a modular way that externalizes all assumptions from the outset, then security reductions could be performed simply by adjusting the boundaries of the adversary. To formalize this concept, SSP introduces the notion of a package, which is a group of oracles with shared state that can invoke the oracles of other packages. For the scheme above, encryption could first be expressed as a composition $\text{MOD}^{\text{OTP}^0, \text{PRF}^0}$ of (1) an OTP package, whose *Enc* oracle outputs ciphertexts; (2) a PRF package, whose *Gen* and *Eval* oracles

output PRF keys and masks (i.e., PRF outputs), respectively; and (3) a wrapper package (MOD) that manages the (order of) calls to other packages. As before, superscript 0 and 1 respectively indicate “real” (non-idealized) and “ideal” functionality. To bound the distinguishing advantage $\text{Adv}(\mathcal{A}^{\text{Real}}, \mathcal{A}^{\text{Ideal}})$, we would again proceed in a sequence of game hops, yet with a different structure, as follows.

Game Real: Oracle encrypting as in Fig. 1. As before.

Game G_1 : Decomposition. A dogmatic state-separating proof would first externalize *all* oracles to be relied on in the proof as *packages*. This leads to an initial transition that establishes perfect indistinguishability of this decomposition, i.e., (slightly oversimplified⁸)

$$\text{Adv}(\mathcal{A}^{\text{Real}}, \mathcal{A}^{(\text{MOD}^{\text{OTP}^0, \text{PRF}^0})}) = 0.$$

With all oracles externalized in this way, successive game hops can mostly be carried out by syntactic matching. However, the hop from G_2 to G_3 requires more than simple matching, since we need to argue (from constraints on the oracle) that the random function is never queried twice on the same input.

Game G_2 : In the Eval oracle, replace PRF by a random function F . This replacement is justified by a reduction from the indistinguishability between the “real PRF package” PRF^0 and the “ideal PRF package” PRF^1 (implementing a truly random function). That is,

$$\text{Adv}(\mathcal{A}^{(\text{MOD}^{\text{OTP}^0, \text{PRF}^0})}, \mathcal{A}^{(\text{MOD}^{\text{OTP}^0, \text{PRF}^1})}) \leq \text{Adv}(\mathcal{B}^{\text{PRF}^0}, \mathcal{B}^{\text{PRF}^1})$$

for $\mathcal{B} := \mathcal{A}^{(\text{MOD}^{\text{OTP}^0})}$.

Game G_3 : Replace PRF^1 by RAND. Here, RAND is a trivial package with an oracle that simply returns a uniformly random mask (i.e., PRF output). Naturally, calling such an oracle is perfectly indistinguishable from calling one that implements a random function (like *eval* of PRF^1). So,

$$\text{Adv}(\mathcal{A}^{(\text{MOD}^{\text{OTP}^0, \text{PRF}^1})}, \mathcal{A}^{(\text{MOD}^{\text{OTP}^0, \text{RAND}})}) = 0.$$

Game G_4 : In the Enc oracle, replace $\oplus$ by uniform sampling. This replacement is justified by an analogous reduction to that in the PRF-based step, but for OTP. That is,

$$\text{Adv}(\mathcal{A}^{(\text{MOD}^{\text{OTP}^0, \text{RAND}})}, \mathcal{A}^{(\text{MOD}^{\text{OTP}^1, \text{RAND}})}) \leq \text{Adv}(\mathcal{B}^{\text{OTP}^0}, \mathcal{B}^{\text{OTP}^1}) = 0$$

for $\mathcal{B} := \mathcal{A}^{(\text{MOD}^{\text{RAND}})}$.

⁸ The attentive reader might have spotted that OTP and PRF are clearly not independent games: The PRF output becomes the OTP key. To express this dependency while maintaining the key reasoning ideas, an SSP-style proof introduces an additional key management package during the decomposition. At this point, the notation above breaks down, and SSP introduces new notation that we also use in Section 4. For now, though, we stick to the Bellare-Rogaway-inspired notation.

Game Ideal: Composition. Similarly to the initial decomposition, we have the following “reverse” composition.

$$\text{Adv}(\mathcal{A}^{(\text{MOD}^{\text{OTP}^1, \text{RAND}})}, \mathcal{A}^{\text{Ideal}}) = 0.$$

Again, we conclude by transitivity of the advantage function that for our adversary $\mathcal{A}$, there exists a PRF adversary $\mathcal{B}$ such that

$$\text{Adv}(\mathcal{A}^{\text{Real}}, \mathcal{A}^{\text{Ideal}}) \leq \text{Adv}(\mathcal{B}^{\text{PRF}^0}, \mathcal{B}^{\text{PRF}^1}).$$

Unlike the Bellare-Rogaway style, the SSP style brings all refactoring steps into a single upfront *decomposition* step, whose structure heavily depends on the considered proof. In return, successive game transitions are made local—anything “irrelevant” can be lumped in with the adversary—and most become trivial since all package boundaries are designed to be adversary boundaries. Nevertheless, the complexity of the initial deconstruction step may be too challenging in practical settings. In most pragmatic SSPs, the deconstruction step is typically interleaved with cryptographic reasoning.

Blended Style. Both extremes outlined above, along with the more pragmatic approaches mentioned, suggests a potential middle ground: one that aims to reduce repeated refactoring (or deconstruction) steps while avoiding an initial deconstruction that requires a complete and precise understanding of the entire proof. In our attempt at this middle ground, which we call the *blended* style or approach, the proof proceeds as follows, using definitions from the SSP style. Since the reasoning is similar to before, just ordered differently, we do not go into too much detail here.

Game Real: Oracle encrypting as in Fig. 1. As before.

Game G_1 : “Externalize” the PRF. For a suitable wrapper package MOD, we show

$$\text{Adv}(\mathcal{A}^{\text{Real}}, \mathcal{A}^{(\text{MOD}^{\text{PRF}^0})}) = 0.$$

Game G_2 : In the Eval oracle, replace PRF by a random function F . As in the SSP-style proof, we have

$$\text{Adv}(\mathcal{A}^{(\text{MOD}^{\text{PRF}^0})}, \mathcal{A}^{(\text{MOD}^{\text{PRF}^1})}) \leq \text{Adv}(\mathcal{B}^{\text{PRF}^0}, \mathcal{B}^{\text{PRF}^1})$$

for $\mathcal{B} := \mathcal{A}^{\text{MOD}}$.

Game G_3 : Replace PRF^1 by RAND. Similarly to the SSP-style proof, we have

$$\text{Adv}(\mathcal{A}^{(\text{MOD}^{\text{PRF}^1})}, \mathcal{A}^{(\text{MOD}^{\text{RAND}})}) = 0.$$

Game G_4 : “Externalize” the OTP. We modify the wrapper package MOD (as MOD') so it never observes the one-time mask that is uniformly-generated by RAND. Instead, the operation of the one-time pad is externalized as a package OTP that relies on RAND to sample its key.

$$\text{Adv}(\mathcal{A}^{(\text{MOD}^{\text{RAND}})}, \mathcal{A}^{(\text{MOD}'^{\text{RAND}, \text{OTP}^0})})$$

Game G_5 : In the Enc oracle, replace $\oplus$ by uniform sampling. Similarly to the SSP-style proof, we have

$$\text{Adv}(\mathcal{A}^{(\text{MOD}'^{\text{RAND}}, \text{OTP}^0)}, \mathcal{A}^{(\text{MOD}'^{\text{RAND}}, \text{OTP}^1)}) \leq \text{Adv}(\mathcal{B}^{\text{OTP}^0}, \mathcal{B}^{\text{OTP}^1}) = 0.$$

for $\mathcal{B} := \mathcal{A}^{(\text{MOD}'^{\text{RAND}})}$.

Game Ideal: Composition. Analogously to before.

In short, the blended approach seeks to externalize packages one by one, without internalizing them between successive games. The pragmatic aim is to avoid refactoring steps and gain in locality, while facilitating step-wise and exploratory reasoning. As part of our experiment, this middle ground allows us to more flexibly assess the difficulties that arise in the two historical styles, by enabling us to remove the definitional and proof features we expect to be problematic. We stress that we *do not* propose or advocate for this approach as an alternative to Bellare-Rogaway-like game-playing or state-separating proofs. Instead, this approach is a purely experimental artifact used to qualitatively test our hypothesis. Further, we use this blended approach (in Section 5) as the basis for further exploration of the proof style space, informed by challenges we did not foresee in the methodology outlined here.

Choice of Tool. We focus on machine-checked cryptographic techniques and proofs [7] for security claims in the computational model (in contrast to the symbolic or Dolev-Yao model [21]). Currently active tools in this area include CryptoVerif [14,1], Squirrel [6,3], SSProve [5,4], and EASYCRYPT [9,8,2]; all focusing on computational security. While CryptoVerif and Squirrel provide security *guarantees* in the computational model, their respective specification languages impose definition styles on the proof writer that deviate significantly from pen-and-paper proofs, and hence are not suitable for our purposes. SSProve was designed specifically for proofs in the style of SSP and hence is only applicable to constructions and claims expressed in this style. EASYCRYPT is an interactive proof assistant designed to formalize code-based, game-playing proofs [11,25]. EASYCRYPT’s specification language is close in style to that proposed by Bellare and Rogaway for specifying code-based games, but adds on a module system making it suitable to express more modular notions as well. In particular, EASYCRYPT has also been used to formalize proofs in the SSP style [22], as well as in the UC framework [19]. It is mainly this flexibility that makes it our choice for the observations made here.

Choice of Case Study. We choose the security proof of **CryptoBox** as a case study because it is known to be valid, and is formalizable in the more restrictive styles we intend to explore. In particular there exists an SSP-style EASYCRYPT proof of **CryptoBox** as nonce-based public-key encryption scheme due to Dupres-soir, Kohbrok and Oechsner [22]. While the goal of their work was to identify a systematic translation of the SSP style into EASYCRYPT, the authors note some

trade-offs observed in their proof: Security proofs in this style require a costly and relatively complex initial proof step that deconstructs the considered scheme into a composition of smaller components from which standard cryptographic assumptions can be expressed, enabling subsequent reduction steps to be done “for free”.

Approach. With the hypothesis and observations from the SSP formalization in mind, we set out to *independently* develop new proofs of the same security result for **CryptoBox** in this work. Our goal is to produce proofs in both the traditional game-playing style and our new blended style. With the same security proof presented in three different styles, we can then compare and contrast the approaches.

Definitions. The traditional game-playing proof uses standard definitions of security for assumptions and goals, whose formalization into EASYCRYPT was discussed between the senior study authors to ensure they were not tainted by the existing SSP-style definitions. The blended approach starts from the existing SSP-style security *notions* (not proofs), as already formalized by Dupressoir, Kohbrok and Oechsner [22]. The starting definitions for all three proofs intend to capture the same notions (see Section 2.2), but *express* them in different ways. Our study outlines some light and reasonable conditions on the equivalence of those expressions.

Proof Authors. The relative rarity of EASYCRYPT users imposes a somewhat imbalanced team: the traditional game-playing proof is written by an experienced—but not quite expert—EASYCRYPT user, while the blended proof is written by a novice EASYCRYPT user with some light guidance. (The original SSP-style proof [22] was developed by an expert EASYCRYPT user.)

Proofs and Evaluation. Both new proofs were developed independently, in distinct repositories and by proof authors at different institutions, with explicit instructions not to refer to the existing SSP-style proof. Once both proofs were complete, the authors wrote independent reports⁹ to describe the definitions, provide an overview of the proof, summarize the challenges faced, and elaborate on the qualitative aspects of the proof’s complexity, with a specific focus on (1) modularity, (2) the need for creativity, and (3) arguments with a significant gap between intuition and formalization. These reports were considered after a unifying pass by another experienced EASYCRYPT user. They also provide a more narrative description of the experience formalizing each proof, and how writing and thinking tasks interleaved, in particular. The findings are summarized in Sections 3 and 5, while Section 4 summarizes observations from the existing SSP-style proof and begins the critical comparison. In Section 5, we reflect on our definitional choices for the second new proof (in blended style), propose small adjustments, and discuss their effect on the proof effort and the strength of the result.

⁹ Provided along with the two new proofs in the associated artifact.

1.2 Observations

The following provides an overview of the main observations and findings from our study, with details deferred to later sections.

Toward a Pragmatic Proof Style. The blended proof style was designed to avoid the formalization difficulties we anticipated based on previous experiences and reports. The ease with which the traditional game-playing proof in this work could be formalized revealed further incompatibilities between SSP-style *definitions* and the formal reasoning as realized in tools like EASYCRYPT. Section 5 discusses these difficulties and incompatibilities, including strategies to address them, taking steps toward making both traditional game-playing and state-separating proofs easier to check and formalize.

Sources of Proof Complexity. From our observations, we identify several sources of proof complexity. We highlight the most important ones here.

Equivalent Games, Supposedly. Occasionally, games that seem trivially or intuitively equivalent turn out not to be, or at least not fully (e.g., the equivalence may rely on some implicit assumption). Such “failures of intuition” incur additional proof complexity when they impose constraints on how components can be used. In the proofs we produce here, this mismatch affects the proofs mostly by affecting their generality, and the ease with which they can be composed. An example we encountered is the issue of sampling keys for multiple parties. Should they be all sampled at the beginning by the experiment, or should the adversary control when each key is sampled? Relating the two versions is possible, but requires reasoning about moving key generation across oracles in the game. This is often justified, on paper, by claiming that a computation is “independent of the adversary’s view”. However, constructing a formal proof of this independence requires careful reasoning and complex invariants. We discuss this in more detail in Section 3.

Reasoning about (Shared) State. Typically, in order to express a reduction, the state of one oracle needs to be split or reproduced between the experiment that represents the algorithmic problem whose hardness we assume, the reduction itself, and the oracles it simulates. Doing so carelessly could yield a malformed reduction (which peeks into the experiment’s state, or into the state of its oracles). This is a relatively common issue to overlook in more complex reductions. In machine-checked proofs, the care with which one must handle such “carving out of state” is reflected in the complexity of the invariants that are used to relate the state of oracles and experiment on one side; and (lower level) oracles, experiment and reduction on the other. These invariants typically then imply equality on the adversary’s view at all points in all executions of the two experiments. In practice *for this proof*, and contrary to Dupressoir, Kohbrohk and Oechsner’s expectations [22], we observe that small but repeated changes to the

state are much less costly in proof effort (using invariant complexity as a proxy) than large changes done once and for all. We discuss these in Section 5, highlighting how they concretely shape the proof efforts. This observation may help focus tool development efforts on pain points specific to the tool’s target proof style.

Over-Restrictive Formal Discipline. We identify and tell a cautionary tale to formal proof authors to avoid dogmatism. In particular, we find that Dupressoir, Kohbrok and Oechsner’s focus on a systematic translation from SSP to EASYCRYPT [22], perhaps combined with the complexity of the usual initial decomposition step in SSP-style proofs, left their proof significantly more complex and slightly more restricted in applicability than necessary. We discuss this further in Section 5.

Simulation Mismatches. The final source of formal complexity reveals an interesting dynamic between modularity and proof effort. Specifically, definitions in traditional game-playing style typically do not capture the operations of corrupt parties in oracles. Instead, they let the adversary carry out the computation should they wish to. For example, encryption under a corrupt key is often not captured by an encryption oracle in traditional definitions. In contrast, SSP-style definitions often attempt to simulate operations of corrupt parties, ensuring that these computations align with those the adversary could perform. This to ensure that a primitive (or package) can be *encapsulated*: a reduction never needs to simulate corrupt computations themselves and can instead rely on the assumption. Although this encourages modular reasoning, it comes at a cost, either in precision or in proof effort. We expand on this in Section 5.

1.3 Contributions

To summarize, our contributions are as follows: (1) We develop two independent code-based, game-playing proofs of security for **CryptoBox** in EASYCRYPT, aiming for the same security notions expressed in different styles, producing independent reports on the formal proofs; (2) we qualitatively compare the proof effort involved in these proofs, and in an existing proof of the same theorem in SSP style. To the best of our knowledge, this is the first direct comparison of traditional game-playing and SSP-style security proofs side by side; (3) based on our experience with developing the different proofs, we identify and discuss qualitative evidence that simply re-expressing security notions equivalently can have an outsize effect on the relative complexity of a machine-checked cryptographic proof; (4) we take steps towards a pragmatic approach to structuring and formalizing code-based, game-playing proofs, aiming for a middle ground between traditional game-playing and state-separating proofs that avoids the formal difficulties associated with both. This small qualitative study is insufficient for us to argue that this approach is *good*.

Some of the lessons learned from this work may be informative to proof authors aiming to minimize the cost of downstream formalization, but also simply

to proof authors aiming to make their pen-and-paper proofs easier to evaluate by a thorough reader.

2 CryptoBox and its Security

Although each proof style leads to a significantly different proof, their fundamental aspects are largely the same, including the considered construction, primitives, and proof intuition. We first go over these shared aspects before diving into the details of the individual proofs.

2.1 CryptoBox: Construction

CryptoBox is a common name for a simple construction combining a NIKE with a particular kind of AE scheme to obtain a public-key version of that encryption scheme with a similar notion of authentication. Nevertheless, we use **CryptoBox** to refer to a more generic (nonce-based) PKAE construction from a NIKE and (nonce-based) AE. The definitions of the relevant schemes and the **CryptoBox** construction are given below.¹⁰

Non-Interactive Key Exchange. A NIKE (with session keys in $\mathcal{K}$, and public and secret keys in $\mathcal{PK}$ and $\mathcal{SK}$) consists of the following two algorithms.

$$\text{KeyGen}() \rightarrow \mathcal{PK} \times \mathcal{SK} \quad \text{SharedKey}(\text{sk} : \mathcal{SK}, \text{pk} : \mathcal{PK}) \rightarrow \mathcal{K}$$

Nonce-Based Authenticated Encryption Scheme (NBAE). An NBAE scheme (with keys in $\mathcal{K}$, nonces in $\mathcal{N}$, plaintexts in $\mathcal{P}$, and ciphertexts in $\mathcal{C}$) consists of the following two algorithms, where $\perp$ is a distinguished indication of failure.

$$\text{Enc}(k : \mathcal{K}, n : \mathcal{N}, p : \mathcal{P}) \rightarrow \mathcal{C} \quad \text{Dec}(k : \mathcal{K}, n : \mathcal{N}, c : \mathcal{C}) \rightarrow \mathcal{P} \cup \perp$$

Instead of including a separate (abstract) **KeyGen**, we define key generation to simply sample uniformly at random from $\mathcal{K}$. (This is because we know we will instantiate it as such to match with the ideal NIKE.)

Nonce-Based Public-Key Authenticated Encryption (NBPKE). An NBPKE scheme (with public and secret keys in $\mathcal{PK}$ and $\mathcal{SK}$, nonces in $\mathcal{N}$, plaintexts in $\mathcal{P}$ and ciphertexts in $\mathcal{C}$) consists of the following three algorithms

$$\begin{aligned} \text{KeyGen}() &\rightarrow \mathcal{PK} \times \mathcal{SK} & \text{Enc}(\text{sk} : \mathcal{SK}, \text{pk} : \mathcal{PK}, n : \mathcal{N}, p : \mathcal{P}) &\rightarrow \mathcal{C} \\ \text{Dec}(\text{sk} : \mathcal{SK}, \text{pk} : \mathcal{PK}, n : \mathcal{N}, c : \mathcal{C}) &\rightarrow \mathcal{P} \cup \perp \end{aligned}$$

CryptoBox. **CryptoBox** is a generic, stateless, non-interactive construction of an NBPKE scheme from a NIKE and an NBAE scheme, as shown in Fig. 2.

¹⁰ A textbook presentation would also include correctness conditions, as well as restrictions on the algorithmic power of each algorithm—whether it can be stateful or probabilistic. Correctness is irrelevant here, and we return to discussions of restrictions on the use of state and probabilistic behaviors in Section 6.

KeyGen()	Enc(sk, pk, n, p)	Dec(sk, pk, n, c)
(pk, sk) $\leftarrow$ N.KeyGen()	$k \leftarrow$ N.SharedKey(sk, pk)	$k \leftarrow$ N.SharedKey(sk, pk)
return (pk, sk)	$c \leftarrow$ E.Enc(k, n, p)	$m \leftarrow$ E.Dec(k, n, c)
	return c	return m

Fig. 2. CryptoBox Construction with NIKE N and NBAE E.

2.2 CryptoBox: Security

Next, we cover the shared aspects and intuition behind the security notions and proof. Our main goal in the remainder of the paper is then to explore and contrast how the different styles formalize these aspects, especially with respect to the gaps they introduce between intuition and the actual proof.

Security Notions. We adopt standard notions of security for NIKE and NBAE, and a natural combination of NIKE’s key management with NBAE’s key usage oracles for NBPKE. We discuss their intuition here, as well as their lineage—useful to understand the biases they would introduce in proofs.

NBPKE. The security notion for NBPKE provides a way to generate honest keys and register corrupt keys (but not to corrupt honest keys), as well as to encrypt and decrypt messages. Parties are identified by their public key, and honest parties are assumed to keep their private key secret (using it only as specified in the scheme). The adversary may inject public keys into the system, which may not have a corresponding valid private key. The adversary can request encryptions of arbitrary messages under the sender’s private key (identified by its public key), and the recipient’s public key. In this case, the sender’s key must be honest while the recipient’s key may be corrupt. Security is defined as an indistinguishability-based notion. Real encryption and decryption oracles compute answers following the scheme’s specification. Ideal encryption oracles output uniformly sampled ciphertext, and ideal decryption oracles fail on all inputs. For security, we require that no nonce-respecting adversary can distinguish real oracles from ideal oracles. The adversary may not observe the output of decryption queries where the ciphertext was produced by an encryption query for the same key and nonce. This notion captures the security model described for the *Curve25519* key exchange and its intended use by Bernstein [12], clarifying that initially proposed by Diffie and Hellman [20].

Security in this context is inherently multi-user: given that the same public key may be used by an honest party in communicating with a number of other parties, some of which may be corrupt and initiate the communication, it is important to ensure that communication between an honest party and a corrupt party does not affect the security of communications between this party and other honest parties. As a consequence, we also adopt multi-user security notions for our primitives.

NIKE. For NIKE security, we adopt the “Simplified” version of Freire, Hofheinz, Kiltz and Paterson’s NIKE notion [24, Appendix G of 2013 revision,¹¹] in its *S-m-CKS* version, which allows arbitrary numbers of honest and corrupt parties, and multiple test sessions. Parties are again identified by their public key. Honest parties can register a public key so long as this key has not been registered before. We further consider an attacker that is able to register (potentially malicious) public keys—but here again, they may not corrupt existing keys—and establish and observe session keys between any pair of registered users, the first of which must be honest. A NIKE scheme is considered secure if, in this setting, no attacker can effectively distinguish session keys between honest parties from uniformly random keys.

NBAE. For authenticated encryption, we adopt a multi-user version of Bellare, Ng and Tackmann’s nonce-based authenticated encryption notion [10] as a baseline. Most notably, we adopt its integrated nature: the adversary’s goal is to distinguish real oracles for encryption and decryption from ideal oracles that sample ciphertexts uniformly for encryptions, and fail on all inputs for decryptions. As with NBPKE, the adversary may not observe the output of decryption queries where the ciphertext was produced by an encryption query for the same key and nonce. We do note, however, that this precise choice of security notion is somewhat irrelevant to the message of this paper: as long as the security of NBPKE and the security of NBAE align in both their intuition and their precise expression, then the proofs will be similar—or indeed identical—to those described here.

Proof Intuition. To better appreciate the subtleties later on, we give a short intuition of the proof for **CryptoBox**’s security (in the above sense). Consider the real NBPKE game with **CryptoBox**, if the NIKE is secure, then an adversary cannot distinguish if the key used for AE is the one we get from NIKE or a random one. Thus, we can replace the key by a random one. However, for a random key, we know that an adversary cannot break the AE scheme; specifically, they cannot distinguish the real and ideal oracles. At this point, we can plug-in the oracles and arrive at the ideal NBPKE game.

2.3 General Notation

Our proof styles are all code-based, with the experiment, scheme and adversary taken together defining a (probabilistic and stateful) program. The subsequent sections can therefore share the following notation. Given a program P with boolean output, we denote with $\Pr[P = 1]$ the probability that the program outputs true when run. The notation extends as expected when the program additionally takes inputs. All of our security notions are indistinguishability properties, defined by a real and an ideal experiment, each parameterized by the scheme and adversary. Given a real experiment *Real* and ideal experiment *Ideal*,

¹¹ Available from <https://eprint.iacr.org/archive/2012/732/20130101:143205>.

a scheme S and an adversary $\mathcal{A}$, we denote the advantage (borrowing notation from Bellare and Rogaway [11]) of $\mathcal{A}$ in distinguishing the two experiments as follows:

$$\text{Adv}_S(\mathcal{A}^{\text{Real}}, \mathcal{A}^{\text{Ideal}}) = |\Pr[\mathcal{A}^{\text{Real}, S} = 1] - \Pr[\mathcal{A}^{\text{Ideal}, S} = 1]|.$$

We omit the scheme when clear from the context.

3 Bellare-Rogaway Style

We outline, then analyze, our first new proof for **CryptoBox**, starting off by laying out Bellare-Rogaway style definitions. Recall that Bellare and Rogaway [11] define security based on a fixed experiment, that runs an **Init** algorithm, then runs the adversary with oracle access to a pre-defined set of oracles, and finally uses a **Finalize** algorithm with input the adversary’s output and the experiment’s state, to decide the experiment’s output. In our case, **Finalize** simply outputs the adversary’s output, and we specify all security notions as a pair of sets of oracles—real and ideal. In oracle descriptions, code with a solid (resp. dashed) outline is only considered in the real (resp. ideal) oracle; code without an outline is considered in both oracles.

3.1 CryptoBox Security Proof

We start by formally expressing the security notions from Section 2.2.

NIKE. Fig. 3 provides the concrete oracles for NIKE.¹² The public key registration and corruption model are implemented through the oracles **RegisterHonest**, **RegisterCorrupt**, and the *registration map* $\mathcal{RM}$ which keeps track of each key’s corruption status (indicated by $\diamond$). Further, because public keys also serve as identities, both honest and corrupt registration ensure uniqueness of these keys throughout. In keeping with the definition of [24] the task of computing shared keys is split between the **CorruptReveal**, for computing keys with at least one corrupt party involved, and **Test** for honest-to-honest key computation. The only difference between the real and ideal oracles is in the **Test** oracle: $\mathcal{O}_N^{\text{RealNIKE}}$ computes the key as normal via **SharedKey**, whereas the $\mathcal{O}_N^{\text{IdealNIKE}}$ generates new random keys for each public key pair. The session id *sid* generated by *gensid*—a commutative operator on public keys—and the *shared key map* $\mathcal{KM}$ are used to uphold a consistent view for the adversary.

NBAE. Defining the NBAE oracles (Fig. 4 and 5) for multi-user security requires refining two aspects which could impact the proof: how to handle (1) key generation and (2) restriction of the adversary’s oracle queries. In their work introducing the notion, Bellare, Ng and Tackmann [10] let the adversary generate new honest instances (and the associated keys) adaptively, registered against

¹² Recall that we only consider static corruption.

$\mathcal{O}_N^{\text{RealNIKE}}$	$\mathcal{O}_N^{\text{IdealNIKE}}$
vars $\mathcal{RM}, \mathcal{KM}$	
Init()	RegisterHonest()
$\mathcal{RM} \leftarrow []$	$(pk, sk) \leftarrow N.\text{KeyGen}()$
$\mathcal{KM} \leftarrow []$	if $pk \notin \mathcal{RM}$:
if $pk \notin \mathcal{RM}$:	$\mathcal{RM}[pk] \leftarrow \diamond$
$\mathcal{RM}[pk] \leftarrow sk$	return true
return pk	return false
return $\perp$	
CorruptReveal(pks, pkr)	Test(pks, pkr)
if $\mathcal{RM}[pks] \neq \diamond \wedge \mathcal{RM}[pkr] = \diamond$:	if $\mathcal{RM}[pks] \neq \diamond \wedge \mathcal{RM}[pkr] \neq \diamond$:
return $N.\text{SharedKey}(\mathcal{RM}[pks], pkr)$	$k \leftarrow N.\text{SharedKey}(\mathcal{RM}[pks], pkr)$
return $\perp$	$sid \leftarrow \text{gensid}(pks, pkr)$
	if $sid \notin \mathcal{KM}$:
	$\mathcal{KM}[sid] \leftarrow \$\mathcal{U}(\mathcal{K})$
	$k \leftarrow \mathcal{KM}[sid]$
	return k
	return $\perp$

Fig. 3. Oracles for NIKE security.

adversary-provided handles with simple validation. However, multi-instance security notions are still commonly defined with an experiment that generates keys for all instances as part of the game’s **Init** procedure. Although both styles are often considered “trivially equivalent” on paper, their differences are more than sufficient to (potentially) significantly impact the formalization effort. For completeness, we consider both styles of key generation using the suffixes “s” (static) and “a” (adaptive) to distinguish them. Both styles use an index/handle-to-key map $\mathcal{KM}$ to keep track of the keys in use. For oracle restrictions, preventing trivial wins, we use explicit nonce and encryption logs to filter queries made to **Enc** and **Dec**. We further handle invalid queries by “ignoring them” and directly returning $\perp$. In the real oracle, valid queries are forwarded to the primitive after looking up the appropriate key. In the ideal oracle, valid encryption queries are answered with random ciphertexts (where we allow the ciphertext to leak the length of the plaintext) and all decryption queries are answered with $\perp$.

NBPKE. The oracles defining NBPKE security, presented in Fig. 6, follow the implementations and reasoning of the NIKE (for corruption modelling) and NPAE oracles (for deciding valid queries).

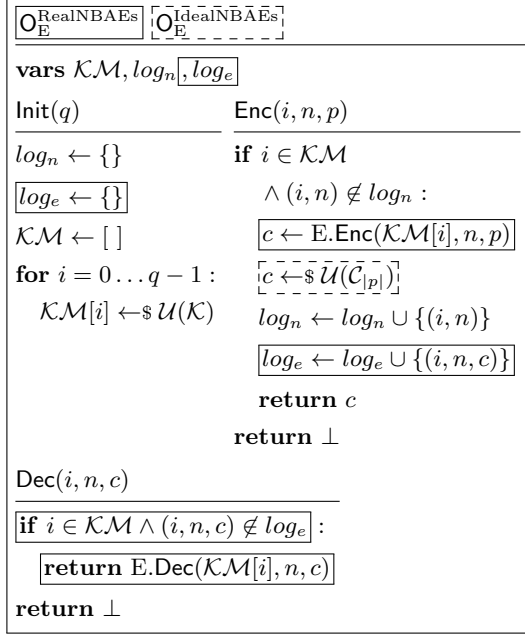


Fig. 4. Oracles for NBAE security with early key sampling.

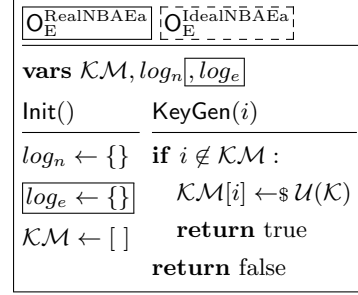


Fig. 5. Oracles for NBAE security with explicit key sampling.

Proof Outline. Due to space constraints, we only provide a proof outline sufficient for our exposition. The formalized proof can be found in the artifact associated with this work. Foremost, we prove the following theorem.

Theorem 1. *For any NIKE scheme N, NBAE scheme E (for which key generation samples a key independently and uniformly at random), and NBPKE adversary $\mathcal{A}$, there exists reductions $\mathcal{R}_0, \mathcal{R}_1, \mathcal{R}_2$ such that the following holds.*

$$\begin{aligned}
 & \text{Adv}(\mathcal{A}^{\text{RealNBPKE}}, \mathcal{A}^{\text{IdealNBPKE}}) \\
 & \leq \text{Adv}(\mathcal{R}_0^{\text{RealNIKE}}, \mathcal{R}_0^{\text{IdealNIKE}}) + \text{Adv}(\mathcal{R}_2^{\text{RealNBAEs}}, \mathcal{R}_2^{\text{IdealNBAEs}})
 \end{aligned}$$

Here, NBAEs reduction $\mathcal{R}_2$ contains NBAEa reduction $\mathcal{R}_1$.

The proof requires a single intermediate game, shown in Fig. 19, which replaces all shared keys that are computed without a corrupted party involved by random keys. We construct $\mathcal{R}_0$ to reduce the NIKE assumption to distinguishing between the original NBPKE game and the intermediate game. Specifically, $\mathcal{R}_0$ simulates the NBPKE encryption and decryption oracles (Fig. 20) by using its own NIKE oracles for shared key generation, and simply performs the necessary AE encryption/decryption itself. Further, it simulates key registration by forwarding queries to the NIKE oracles, and keeping track of corruption status to select the appropriate NIKE oracle for shared key generation.

Where a pen-and-paper proof might go straight for a final reduction to NBAEs (with early key generation), formalization makes this step extremely

$\mathcal{O}_S^{\text{RealNBPKE}}$	$\mathcal{O}_S^{\text{IdealNBPKE}}$
vars $\mathcal{RM}, \log_n, \log_e$	
Init()	RegisterHonest()
$\mathcal{RM} \leftarrow []$	$(pk, sk) \leftarrow S.\text{Keygen}()$
$\log_n \leftarrow \{\}$	if $pk \notin \mathcal{RM}$:
$\log_e \leftarrow \{\}$	$\mathcal{RM}[pk] \leftarrow \diamond$
$\mathcal{RM}[pk] \leftarrow sk$	return true
return pk	return false
return $\perp$	
Enc (pks, pkr, n, p)	Dec (pkr, pks, n, c)
sid $\leftarrow$ gensid(pks, pkr)	sid $\leftarrow$ gensid(pks, pkr)
if $\mathcal{RM}[pks] \neq \diamond \wedge pkr \in \mathcal{RM}$	if $\mathcal{RM}[pkr] \neq \diamond \wedge pks \in \mathcal{RM}$
$\wedge \text{sid} \notin \log_n$:	$\wedge (\text{sid}, n, c) \notin \log_e$
$c \leftarrow S.\text{Enc}(\mathcal{RM}[pks], pkr, n, p)$	$\wedge \mathcal{RM}[pks] = \diamond$:
$\text{if } \mathcal{RM}[pkr] = \diamond$:	return S.Dec($\mathcal{RM}[pkr]$, pks, n, c)
$c \leftarrow S.\text{Enc}(\mathcal{RM}[pks], pkr, n, p)$	return $\perp$
else : $c \leftarrow \$\mathcal{U}(\mathcal{C}_{ p })$	
$\log_n \leftarrow \log_n \cup \{(i, n)\}$	
$\log_e \leftarrow \log_e \cup \{(i, n, c)\}$	
return c	
return $\perp$	

Fig. 6. Oracles for NBPKE security.

unintuitive. We isolate the source of difficulty by first carrying out a reduction $\mathcal{R}_1$ (Fig. 21) from NBPKE, which trivially simulates the NBPKE registration oracles. To simulate the encryption and decryption oracles, $\mathcal{R}_1$ takes over the entire computation for queries that involve a corrupt public key, and simply relies on the NBPKE oracles for honest computations. Note that re-registering a handle that has already been registered is defined as a no-op.

For the final reduction, which relates NBPKE and NBPKEs, we prove a generic lemma.

Lemma 1. *For any NBPKE scheme E (for which key generation samples a key independently and uniformly at random) and NBPKE adversary $\mathcal{B}$ that makes at most $q \in \mathbb{N}$ queries to its **KeyGen** oracle, there exists a reduction $\mathcal{R}_2$ such that the following holds.*

$$\text{Adv}(\mathcal{B}^{\text{RealNBPKE}}, \mathcal{B}^{\text{IdealNBPKE}}) = \text{Adv}(\mathcal{R}_2^{\text{RealNBPKEs}}, \mathcal{R}_2^{\text{IdealNBPKEs}})$$

The reduction $\mathcal{R}_2$ (Fig. 22) maintains a map between handles and indices that enables indirect addressing of the pre-sampled keys. Additionally, the conditions on

forwarding encryption/decryption queries prevent non-registered handles from being used. Since the keys are set once and never modified, this is sufficient for $\mathcal{R}_2$ to simulate the NBAEa queries.

Combining Lemma 1 with the earlier reductions, we obtain the desired bound, establishing the security of `CryptoBox`(N, E). Remark that, for Lemma 1 to apply, $\mathcal{R}_1$ should issue at most q queries to its `KeyGen` oracle. By construction, this holds if (and only if) $\mathcal{A}$ issues at most q queries to its `Enc` and `Dec` (combined).

3.2 Analysis and Observations

When formalizing the proof in EASYCRYPT, there is a gap between the pen-and-paper arguments and their formalization. For example, consider the first game hop above: it applies the NIKE assumption by finding a reduction and justifying soundness through an intuitive explanation of how the reduction simulates the necessary oracle calls. This intuition must be captured rigorously in formal proofs using *program invariants*—specifications on the state of the oracles and the game that hold true throughout the execution of the adversary.¹³ They have two main forms: relational and non-relational. Relational invariants refer to the relation between two program states, often used to show equivalence or other relationships between games. Non-relational invariants focus on a single program state and are useful when having to analyze more complex control flow sequences in a single game. For the NIKE reduction, the invariants on the real side are straightforward: the logs and registration map maintained by the reduction adversary (Fig. 20) must be consistent with their counterparts in the RealNBPKE game (Fig. 6). These invariants are relatively easy to argue within the formalization.

Although the pen-and-paper arguments for the first game hop were relatively simple to formalize, this is not always the case. For Lemma 1, the formal proof of the reduction’s soundness requires extensive use of so-called lazy/eager arguments—a mechanism to delay sampling values in a game, often just before their use. In our case, we delay sampling of each AE key from the `Init` oracle to the `KeyGen` oracle. Here, the hidden complexity is that moving sampled values across oracle boundaries requires extensive reasoning in EASYCRYPT: we must be certain that these values are not dependent on any of the computation that oracles could perform during the adversary’s execution. Fortunately, since this is done as an extra reduction step, we reduce the combinatorial load by considering just the NBAE games rather than the entire context of the NBPKE games. So, this abstraction not only simplifies the intuition, but also the formalization effort. Moreover, the EASYCRYPT standard library provides proofs for more general lazy-eager arguments, which can be instantiated and reused for our specific proof with some additional work. Together, these simplifications

¹³ In our case, the game is defined entirely in terms of oracles, so their state captures everything.

and abstractions turn what would have been an incredibly challenging task into “just” a challenging task.¹⁴

One important observation is that choosing NBAEa over NBAEs as our NBAE definition circumvents the complex argument above entirely. Since NIKE security, and thus the NBPKE security, studies a protocol-like object, we expect some interaction to precede the establishment of shared keys. In contrast, for primitives like NBAE, the timing and method of key generation is not as important, leading to the two choices we have seen.

The potential for lack of composition between definitions is one problem that SSP purports to address nicely. Looking forward, we revisit the SSP of CryptoBox and examine exactly how well this problem is solved.

4 State-Separating Proof Style

We now revisit the same security proof discussed in the preceding section, yet in the style of SSP. The corresponding EASYCRYPT proof was previously developed by Dupressoir, Kohbrok, and Oechsner [22], and we will briefly recap it moving on to the analysis.

4.1 State-Separating Proofs

As discussed in Section 1.1, SSP is a methodology for writing security proofs due to Brzuska, Fournet, Delignat-Lavaud, Kohbrok, and Kohlweiss [17]. The stated goal of SSP is to provide a more compositional approach to definitions and reasoning. Recall that in Bellare and Rogaway’s framework, a game is defined as a set of oracles sharing some (local) state. The adversary interacts with these oracles but cannot directly observe the game’s local state. SSP generalizes this by introducing the notion of a *package*, which consists of oracles with shared local state that can also invoke oracles from other packages. In this context, a *game* is a special case of a package that does not make calls to oracles of any other package (oracles are allowed to call other oracles in their package). An *adversary* is a special package with a single abstract oracle that returns a bit. Packages can be composed sequentially and in parallel, and package composition is associative. SSP-style security notions are typically phrased as indistinguishability games, and the proofs use the same general techniques as in traditional game-playing: code transformations, applications of the failure event lemma [25] to establish equivalence up to a bad event, and reductions from the security of components or computational hardness assumptions. In addition, the algebraic properties of SSP package composition enable decomposing arguments about a game into arguments about sub-games.

¹⁴ For context, this step requires more lines of proof than all other steps combined when proving in EASYCRYPT.

vars $\mathcal{PK}, \mathcal{SK}$		
Gen()	Csetpk(pk)	Getsk(pk)
(pk, sk) $\leftarrow$ \$ dkg if $\mathcal{PK}[\text{pk}]$: return $\perp$ return $\perp$ $\mathcal{PK}[\text{pk}] \leftarrow \text{true}$ $\mathcal{SK}[\text{pk}] \leftarrow \text{sk}$ return pk	if pk $\in \mathcal{PK}$:	if pk $\notin \mathcal{SK}$:
	return $\perp$ $\mathcal{PK}[\text{pk}] \leftarrow \text{false}$	return $\perp$ return $\mathcal{SK}[\text{pk}]$ Honpk(pk) if pk $\notin \mathcal{PK}$: return $\perp$ return $\mathcal{PK}[\text{pk}]$

Fig. 7. Oracles of PKEY_{dkg} .

4.2 CryptoBox Security Proof

Before proceeding to the actual proof, we present the SSP-style security notions for NBPKE, NIKE and NBE used by Dupressoir, Kohbrok and Oechsner [22]. Each security notion consists of two games, and each game is in turn described as composition of multiple packages. Some packages appear in more than one game, which is relevant for the security proof in Section 4.2.

NBPKE. The NBPKE assumption is expressed as indistinguishability of two games GPKE_P^b , parameterized by an NBPKE scheme P and a bit b . The real game GPKE_P^0 encrypts and decrypts according to P while the ideal game GPKE_P^1 performs log-based ideal encryption and decryption for honest keys. Each game is defined as composition of two packages, PKEY_{dkg} and PKE_P^b . The PKEY_{dkg} package (Fig. 7) is parameterized by a distribution over key pairs (dkg) and stores keys as well as their corruption status. Each key is identified by its public key pk. The package uses internal state $\mathcal{SK}$ and $\mathcal{PK}$ to store a mapping of secret keys to public keys and the honesty status of public keys, respectively. Honest keys can be generated and stored by calling oracle **Gen**, and corrupt public keys can be stored via **Csetpk**. Note that **Gen** fails if a freshly sampled honest public key collides with an existing corrupt public key.

Packages are often represented as graphs like shown in Fig. 8. In such graphs, a box represent a package (e.g. PKEY_{dkg}), and the connected arrows represent the package's oracles (e.g. **Gen**, **Csetpk**, **Getsk**, **Honpk**). An incoming arrow indicates that the package's oracles can be called by the oracles of another package (i.e., the one corresponding to the box from which the arrow originates). Since PKEY_{dkg} does not call any other oracles, it is technically a game, albeit not a particularly useful one at this point. An adversary is simply a special package, so it can be composed with a game to give the adversary black-box access to the game's oracles. In Fig. 8, the adversary package would appear to the "left". In line with

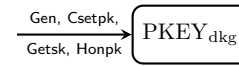


Fig. 8. Package graph for PKEY_{dkg} .

the SSP literature, we omit adversary packages from package graphs and simply assume that there is an adversary with access to all “visible” oracles of the package.

vars $\mathcal{M}$	
Pkenc ($\text{pk}_s, \text{pk}_r, m, n$)	Pkdec ($\text{pk}_r, \text{pk}_s, c, n$)
$\text{sk}_s \leftarrow \text{GetSk}(\text{pk}_s)$	$\text{sk}_r \leftarrow \text{GetSk}(\text{pk}_r)$
$\text{hon}_{\text{pk}_r} \leftarrow \text{HonPk}(\text{pk}_r)$	$\text{hon}_{\text{pk}_s} \leftarrow \text{GetSk}(\text{pk}_s)$
$h \leftarrow \text{gensid}(\text{pk}_s, \text{pk}_r)$	if $b \wedge \text{hon}_{\text{pk}_s}$:
if $(h, n) \in \mathcal{M}$:	$h \leftarrow \text{gensid}(\text{pk}_s, \text{pk}_r)$
return $\perp$	$m \leftarrow \text{getmsg}(\mathcal{M}[h, n], c)$
if $b \wedge \text{hon}_{\text{pk}_r}$:	else $m \leftarrow \text{P.Pkdec}(\text{sk}_r, \text{pk}_s, c, n)$
$c \leftarrow \$ \mathcal{D}_c(m)$	return m
else $c \leftarrow \text{P.Pkenc}(\text{sk}_s, \text{pk}_r, m, n)$	
$\mathcal{M}[h, n] \leftarrow (m, c)$	
return c	

Fig. 9. Oracles of the PKAE_P^b package. Here, gensid generates a (unique) handle for each pair of public keys; $\mathcal{D}_c$ denotes the ciphertext distribution of P (accounting for message length); and $\text{getmsg}((m, c'), c)$ returns m if $c = c'$ and $\perp$ otherwise.

The package PKAE_P^b (Fig. 9) is parameterized by an NBPKE scheme P and provides oracles for encryption and decryption according to P . Note that its oracles Pkenc and Pkdec call oracles GetSk and HonPk of another package. That is, the package PKAE_P^b does not sample or manage any keys itself. The package is also parameterized by an *idealization bit* b determining the behavior of the oracles: The real version ($b = 0$) performs encryption and decryption as per P , while the ideal version ($b = 1$) performs a log-based version instead. Fig. 10 shows the graphical representation of PKAE_P^0 . The package provides oracles Pkenc and Pkdec , represented by incoming arrows. The arrows pointing from PKAE_P^0 to the outside represent calls to oracles GetSk and HonPk of another package.



Fig. 10. Package graph for PKAE_P^0 .

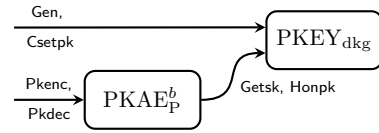


Fig. 11. Package graph for GPKE_P^b .

As shown in Fig. 11, we obtain the NBPKE security game GPKE_P^b by composing PKAE_P^b with the key package PKEY_{dkg} , i.e. map the calls of GetSk and HonPk in PKAE_P^b to the corresponding oracles of PKEY_{dkg} . Note that the

vars $\mathcal{H}, \mathcal{K}$		
$\text{Set}(h, k)$	$\text{Cset}(h, k)$	$\text{Get}(h)$
if $h \in \mathcal{K}$:	if $h \in \mathcal{K}$:	if $h \notin \mathcal{K}$:
return $\perp$	return $\perp$	return $\perp$
$\mathcal{H}[h]y \leftarrow \text{true}$	$\mathcal{H}[h] \leftarrow \text{false}$	return $\mathcal{K}[h]$
if b :	$\mathcal{K}[h] \leftarrow k$	Hon(h)
$\mathcal{K}[h] \leftarrow k$		if $h \notin \mathcal{H}$:
else :		return $\perp$
$\mathcal{K}[h] \leftarrow \$ dk$		return $\mathcal{H}[h]$

Fig. 13. Oracles of the KEY_{dk}^b packages.

$\text{Sharedkey}(\text{pk}_s, \text{pk}_r)$
$\text{hon}_{\text{pk}_s} \leftarrow \text{Honpk}(\text{pk}_s)$
$\text{hon}_{\text{pk}_r} \leftarrow \text{Honpk}(\text{pk}_r)$
$h \leftarrow \text{gensid}(\text{pk}_s, \text{pk}_r)$
if $\text{hon}_{\text{pk}_s} \wedge (\text{hon}_{\text{pk}_r} \neq \perp)$:
$\text{sk}_s \leftarrow \text{Getsk}(\text{pk}_s)$
$k \leftarrow \text{N.Sharedkey}(\text{pk}_r, \text{sk}_s)$
if $\text{hon}_{\text{pk}_r} : \text{Set}(h, k)$
else : $\text{Cset}(h, k)$

Fig. 14. Oracle of NIKE_N package.

new game no longer provides these oracles as they have become internal oracle calls available to PKAE_P^b only. Previously, in Section 1.1, we denoted package composition in superscript notation. For the remainder, package composition is described through graphs to capture more complex composition patterns.

NIKE. The NIKE assumption is expressed as indistinguishability of two games GNIKE_N^b , parameterized by a NIKE scheme N and an idealization bit b . The real game GNIKE_N^0 outputs a shared key in accordance with N , while the ideal game GNIKE_N^1 outputs a random key. Each game is formed by composing three packages: PKEY_{dkg} , which we saw above, and two new packages, NIKE_N and KEY_{dk}^b . This composition is depicted in Fig. 12, where $\text{dkg} = N.\text{dkg}$. The key management package KEY_{dk}^b is shown in Fig. 13. The package stores (symmetric) keys and their corruption status. KEY_{dk}^b is parameterized by a key distribution dk , and idealization bit b . Each key is identified by a handle h . The package uses local state $\mathcal{K}$ to track the mapping of existing keys to handles, as well as $\mathcal{H}$ to track the honesty status of keys. Keys can be stored by calling oracles Set and Cset . Set marks a key as honest; Cset marks a key as corrupt. In the real version of the package ($b = 0$), Set resamples and stores a fresh key from dk . This is an SSP modeling trick to encode the sampling and setting of keys from the outside in a uniform way.

The NIKE_N package (Fig. 14) is parameterized by a NIKE scheme N and provides a single oracle Sharedkey . The oracle takes as input two public keys pk_s, pk_r and derives a shared secret key k from them using N . It then stores k together with its corruption status in package KEY_{dk}^b . Note that the handle h is derived deterministically from pk_s and pk_r .

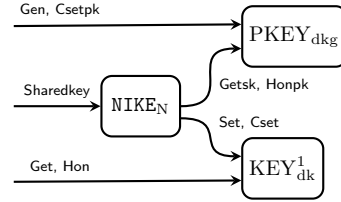


Fig. 12. Package graph for GNIKE_N^1 .

NBAE. The NBAE assumption is expressed as indistinguishability of two games GAE_E^b , parameterized by an NBAE scheme E and an idealization bit b . The games are defined as composition of the KEY_{dk}^1 package from before and a new AE_E^b package. (Fig. 23 and 24) The NBAE package AE_E^b works similarly to the NBPKE package in Fig. 9.

Proof Outline. We are only interested in the part of the security proof that was named “core lemma” in [22, Lemma 4]. The security statement itself looks as expected.

Lemma 2 (Core Lemma [22]). *Let E be an NBAE scheme and let N be a NIKE. Then, for any adversary $\mathcal{A}$, there exist reductions $\mathcal{R}_{\text{GNIKE}}$ and $\mathcal{R}_{\text{GAE}}$ (with time complexity similar to that of $\mathcal{A}$) such that*

$$\epsilon_{\text{GPKAE}}^{\text{CryptoBox}(\text{N}, \text{E})}(\mathcal{A}) \leq \epsilon_{\text{GNIKE}}^{\text{N}}(\mathcal{A}^{\mathcal{R}_{\text{GNIKE}}}) + \epsilon_{\text{GAE}}^{\text{E}}(\mathcal{A}^{\mathcal{R}_{\text{GAE}}})$$

where $\epsilon_G^S(\mathcal{B})$ denotes the distinguishing advantage of adversary $\mathcal{B}$ in game G with respect to scheme S .

At a high level, the security proof follows the expected outline: first reduce from the NIKE assumption, then reduce from the NBAE assumption. The key difference between the SSP and traditional game-playing style is exactly in how the reductions are performed. Recall that, in the traditional game-playing style, the proof proceeds in a sequence of game hops. When a game hop from, say, G^1 to G^2 is shown via a reduction from an assumption, first the two games are rewritten in terms of the reduction and the assumption, then soundness of the reduction is shown, and finally the assumption is applied. Importantly, the reduction is constructed specifically for this proof step. That is, we simply “forget” about the reduction after this point. The SSP methodology proposes another approach: Instead of constructing a new reduction to some assumption at the start of each game hop, the strategy is to start the overall security proof by a single large game hop that rewrites the entire game into a modular version. This *decomposed* game is carefully crafted such that any assumption H^b and reduction $\mathcal{R}$ —needed during the remaining game hops—are either explicitly contained as sub-graph, or appear after applying assumptions. With the decomposed game set up, subsequent reductions are merely syntactic rewriting: Identify the assumption to be applied as composition of some packages, and define the reduction to be the rest.

For Lemma 2, the SSP strategy is depicted in Fig. 15.

Decomposition. The SSP proof starts by decomposing the real NBPKE game $\text{GPKAE}_{\text{CryptoBox}(\text{N}, \text{E})}^0$ as shown in Fig. 15. The $\text{PKAE}_{\text{CryptoBox}}^0$ package is split into multiple packages. NIKE_N^0 and AE_E^0 capture **CryptoBox**’s use of the NIKE and NBAE scheme, respectively. KEY_{dk}^0 manages the shared state between the NIKE and NBAE scheme, i.e. the shared secret key that is output by the NIKE scheme to become an input for the NBAE scheme. Finally, a new package **MODPKE** simulates **CryptoBox** by calling the other packages in the appropriate order.

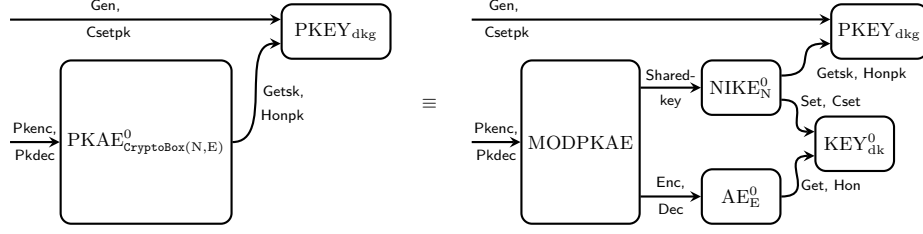


Fig. 15. Decomposing real game $\text{GPKAE}_{\text{CryptoBox}(N,E)}^0$ (left) into modular version (right).

NIKE Reduction. Note that the modular game contains the real NIKE game GNIKE_N^0 as sub-graph. The remaining packages can simply be *defined* to be the reduction $\mathcal{R}_{\text{GNIKE}}$ as shown in Fig. 16, which immediately implies reduction soundness. At this point, we can trivially idealize by applying the NIKE assumption.

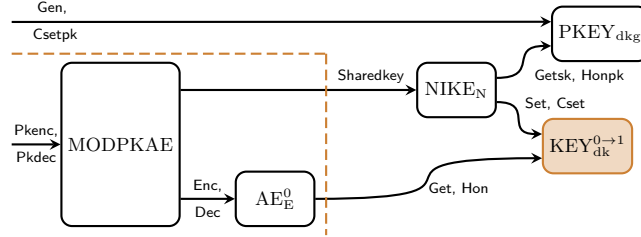


Fig. 16. NIKE reduction: Game GNIKE_N^b (right of dashed line) and reduction $\mathcal{R}_{\text{GNIKE}}$ (left of dashed line).

NBAE Reduction. After applying the NIKE assumption, KEY_{dk}^0 has been replaced by KEY_{dk}^1 , and the real NBAE game GAE_E^0 has appeared as sub-graph. Again, we can define an NBAE reduction $\mathcal{R}_{\text{GAE}}$ as a composition of existing packages (see Fig. 25) and directly apply the NBAE assumption to idealize.

Reconstruction. The initial decomposition has been set up precisely to reach a modular game that is equivalent to the ideal $\text{NBPKE}_{\text{CryptoBox}(N,E)}^1$. It merely remains to prove the reconstruction that reverses the decomposition (as shown in Fig. 26).

4.3 Analysis and Observations

For the SSP-style proof, we identify several sources of proof complexity. We discuss the main ones here, drawing comparisons with the Bellare-Rogaway-style proof discussed in Section 3 where relevant.

Invariant Complexity. Consider the reduction from NIKE security, which requires two steps in the SSP proof: a decomposition step and a syntactic rewrite with the NIKE assumption. The latter is trivial, which is one of the key selling points of SSP. The former, however, is much more involved than one might expect when first encountering a proof sketch. Here, the complexity arises from the need to relate the state of the two games to argue their indistinguishability. This is done through invariants.

The first type of invariant relates the parts of the state that are meant to be identical in both games: (1) the adversary’s state; (2) the public key information $\text{PKEY}.\mathcal{SK}/\text{PKEY}.\mathcal{PK}$; and (3) the message logs of the encryption schemes P and E. However, additional invariants are needed. Following the SSP style, when transitioning from the monolithic game to the modular version with multiple packages, part of the state is split across these packages. Moreover, extra state is required to maintain consistency between the split state. For example, the PKAE^0 package does not store shared keys but instead relies on **CryptoBox** (i.e. its scheme P) to recompute them. In contrast, the modular version of the game introduces a separate package KEY^b for the management and storage of shared keys. Consequently, a second type of invariant, concerning the local state of the decomposed game, arises. This invariant is more complex, conceptually stating the following: (1) consistency between $\text{PKEY}.\mathcal{SK}/\text{PKEY}.\mathcal{PK}$ and $\text{KEY}^0.\mathcal{K}/\text{KEY}^0.\mathcal{H}$ (e.g., if a public key pk is corrupted, so is any shared key derived from pk); and (2) consistency between $\text{KEY}^0.\mathcal{K}/\text{KEY}^0.\mathcal{H}$ and $\text{AE}^0.\text{log}$ (e.g., if no shared key exists for handle h , then no ciphertext is recorded in the log for h).

Encapsulation of Adversarial Computation. A notable difference between the SSP style and the traditional game-playing style is the extent to which the assumptions encapsulate the behavior of a scheme, even in the case of corruption. Consider the NBAE games from Section 4.1 as an example. The encryption oracles explicitly handle the case of encryption queries under *corrupt* keys. Even in the ideal game, the encryption oracle ensures that encryption queries for *corrupt* keys are answered with honestly computed ciphertexts. As a principle, SSP seeks to align the interface of a game with the interface of the primitive it describes. In contrast, the corresponding NBAE game in the traditional game-playing style simply ignores the case of corrupt keys, as the adversary can compute the corresponding ciphertexts themselves.

The SSP style of explicitly capturing adversarial computation has the benefit of leading to greater modularity. This means that, when using a primitive as part of a bigger construction composed of smaller primitives, the security notions can be composed in the same way; there is no missing “behavior” that would need to be simulated. An example of this simulation in the traditional game-playing proof is the reduction from NBAE security (Fig. 21), which forwards honest encryption queries to an NBAE oracle, but answers corrupt encryption queries itself. However, this choice of encapsulating adversarial computation in SSP may complicate the invariants (discussed above), potentially increasing proof complexity further.

5 Finding Middle Ground

Next, we explore the middle ground between SSP and game-playing techniques. Instead of presenting a single approach, we introduce a spectrum of strategies that fall within an optimal range between these two methods. We derive different proofs along this spectrum, each offering varying levels of modularity.

5.1 Stepwise SSP Style

We start from the SSP in [22], adhering to the same definitions for the security assumptions while addressing the challenge posed by its complex deconstruction step. This step concentrates much of the effort before the “actual” proof, making it cumbersome and less intuitive for cryptographers to follow. To find a middle ground, we applied SSP concepts in a more practical and intuitive manner, gradually breaking the game into modular components, which allows for smaller, more manageable steps throughout the proof process. We maintain an interface-based definition style to preserve modularity and support local reasoning in this proof. Additionally, this approach enables the use of graphical representations and facilitates the mapping SSP concepts to EASYCRYPT’s module system, as discussed in [22].

Modifications and Proof Outline. Rather than proving the equivalence to the complete decomposition of the game, each step focuses on deconstructing only the part we wish to idealize and reason about. For this proof, illustrated in Fig. 27, the first step involves isolating the NIKE game. Next, before extracting the NBAE game, we apply the NIKE assumption and replace the real key management package with the ideal one. Once the NBAE game is established, we can idealize it as well, ultimately achieving the same deconstructed ideal game as in the SSP-style proof.

While planning this proof with graphical representations, we observed that the stepwise method simplifies the deconstruction process. However, the underlying definitions can lead to avoidable proof complexity. In particular, the NIKE assumption as formulated in the SSP approach requires an invariant on the value of keys. This is because the real NIKE game stores computed keys once and retrieves them, rather than recomputing them for every query. This creates a gap between the intuition that a handle consistently references the same key and the formal assertion that all stored keys are correctly generated. While this gap exists in the SSP style, it can be avoided in our approach by being less dogmatic and allowing for an asymmetrical assumption.

Concretely, in contrast to the SSP approach, we allow the topology of our graphs on both sides to differ, leading to a different NIKE assumption. While the ideal side remains unchanged, the real side of our game consists only of a PKEY and a NIKE⁰ package, as shown in Fig 17. In the real NIKE⁰ package, we use an internal log to track the state of keys, rather than depending on an external package. The main distinction is that NIKE⁰ recalculates the key and

overwrites the state for every query. Additionally, this modification relocates the `Get` and `Hon` interfaces to the NIKE^0 package, where they function similarly to how they did in the KEY^0 package.

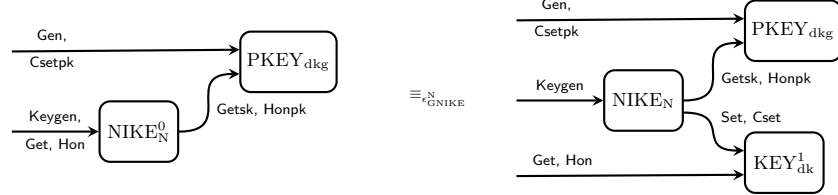


Fig. 17. Modular description of GNIKE_N^0 (left) and GNIKE_N^1 (right), defined as in the SSP style.

With this modified assumption, the proof invariants simplify to equivalences of state. As a result, the two steps required for deconstruction mainly involve rearranging code, making them trivial. In fact, these steps can be proven using extremely basic (built-in) reasoning in `EASYCRYPT`. However, to fully prove a real/ideal security statement, as shown in Fig. 28, we must reconstruct the ideal game facing the same issues as in the SSP-style proof. These issues primarily arise because we need to argue about functional correctness, stemming from the fact that the AE package retrieves the shared key from a state, rather than recomputing it in one of its branches. Instead of proving the same fine-grained modularity¹⁵ as the SSP style at the cost of proving this more complex invariant, we settle for a simulation-based security claim achieved with low effort. This is a stronger security notion leading to a loss of expressiveness and a coarser modularity.

5.2 Blended Style

With both the stepwise-style and traditional game-playing style proofs at our disposal, we explore a middle ground that allows a fine-grained argument without the complexity of the reconstruction step. As noted earlier, the main challenges in SSP-based proofs arise from replacing computation with state, and invariants related to functional correctness. To bypass this issue, we avoid handling cases where state retrieval is necessary—specifically, when the receiver is corrupt and the adversary can perform the computation itself.

Modifications and Proof Outline. The approach described above leads to a modification of the AE package that results in a loss of modularity: any protocol integrated with this revised AE package is required to manage the encryption log internally. The corresponding modified NBAE assumption only performs

¹⁵ We use fine-grained vs. coarse-grained modularity as terms to distinguish between the real/ideal and simulation-based notions.

computations when both public keys involved in a query are honest, similar to the traditional game-playing definitions. However, unlike the traditional game-playing approach, it still supports multiple queries on the same session and nonce, ensuring consistent values rather than failing.

As shown in Fig. 18, the NBAE game still includes both an AE and a KEY package. However, since we apply the assumption after idealizing the keys, we only consider the KEY^1 package. This package allows the adversary to **Set** keys, which are resampled and stored for sessions that have no registered shared key yet. Additionally, the KEY^1 package provides these keys to the AE^b package via **Get** queries. The **Hon**

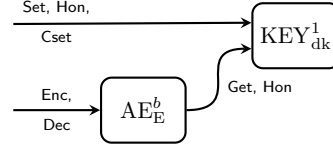


Fig. 18. Package graph for GAE_E^0 .

interface is accessible to both AE *and* the adversary, since it is needed for the reduction but does not reveal additional information. Importantly, AE is guaranteed to use only honest keys, allowing us to maintain consistency with the NIKE assumption by retaining the **Cset** interface. The AE^b package provides **Enc** and **Dec** interfaces that eliminate computations in the corrupt setting by checking the returned value of **Hon**. Otherwise, encryption and decryption are performed as before.

The formal theorem we prove is equivalent to Lemma 2 considered in the SSP-style proof. The proof, depicted in Fig. 29, follows a three-step process: (1) externalize the NIKE game, (2) externalize the NBAE game, and (3) reconstruct the ideal NBPKAE game. The first two steps correspond directly to the stepwise proof presented earlier. In the final step, we still require invariants to ensure that honesty is treated consistently on both sides. Compared to the SSP-style proof, though, these invariants are purely relational, which reduces the complexity of the proof since these do not require reasoning about functional correctness. Moreover, instead of simulating corruption, we let the adversary carry out computations, further simplifying the proof.

6 Conclusion

Our study shows that the style in which definitions are expressed has some effect on the ease of formalization in EASYCRYPT. We have discussed a number of significant differences in proof complexity between the different styles of game-playing definitions, and between systematic and pragmatic approaches to expressing SSP-style definitions. For this particular proof, we also identify a possible trade-off in proof effort depending on whether the chosen definitional style lends itself well to modular and compositional reasoning. While we don't intend our observations to be read as general recommendations, we hope they can benefit future formalization efforts (be it in EASYCRYPT or other tools) in choosing an appropriate proof style.

All Due to EASYCRYPT? We argue that the difficulties we highlight are not *only* due to the formalization, but in fact also hint at some deeper concerns. Recall, from our preliminaries, that we did not impose restrictions on the algorithmic components of our schemes. In practice, a NIKE’s keypair or shared key generation algorithms may make use of an eXtensible Output Function (XOF), typically modelled as a lazily-sampled random oracle, which the adversary can also query. Such algorithms are deeply stateful and probabilistic, and we *want* our proofs—even when done with pen and paper—to apply to such schemes. In such a setting, arguing that the precise interleaving of keypair generations, XOF queries, and computations of shared keys is irrelevant to the adversary’s view requires, in general, a deep semantic argument. In our study, this is reflected by the need, in the SSP and our original blended approach, to assume that the NIKE’s *SharedKey* algorithm is a pure mathematical function: Indeed, any SSP, even carried out with pen and paper, does not apply to instantiations where the NIKE relies on a random oracle or XOF to generate shared secrets.

Next Steps. This study makes it clear that there is *something* to investigate further: the style in which definitions and proofs are expressed and carried out does have some effect—even on a proof as simple as the one we consider here. We have argued that these differences are not solely attributable to the tool we used, or even to the fact that we used a tool. We hope that the observations we make here can serve as starting point for broader studies including other tools, models (e.g., UC and symbolic model), and larger protocols, as well as involving a wider range of proof authors.

References

1. Cryptoverif: Cryptographic protocol verifier in the computational model. <https://bblanche.gitlabpages.inria.fr/CryptoVerif/>.
2. Easycrypt. <https://easycrypt.info>.
3. Squirrel prover. <https://squirrel-prover.github.io/>.
4. Ssprove. <https://github.com/SSProve/ssprove>.
5. Carmine Abate, Philipp G. Haselwarter, Exequiel Rivas, Antoine Van Muylder, Théo Winterhalter, Catalin Hritcu, Kenji Maillard, and Bas Spitters. SSProve: A foundational framework for modular cryptographic proofs in coq. In Ralf Küsters and Dave Naumann, editors, *CSF 2021 Computer Security Foundations Symposium*, pages 1–15. IEEE Computer Society Press, 2021.
6. David Baelde, Stéphanie Delaune, Charlie Jacomme, Adrien Koutsos, and Solène Moreau. An interactive prover for protocol verification in the computational model. In *2021 IEEE Symposium on Security and Privacy*, pages 537–554. IEEE Computer Society Press, May 2021.
7. Manuel Barbosa, Gilles Barthe, Karthik Bhargavan, Bruno Blanchet, Cas Cremers, Kevin Liao, and Bryan Parno. SoK: Computer-aided cryptography. In *2021 IEEE Symposium on Security and Privacy*, pages 777–795. IEEE Computer Society Press, May 2021.
8. Gilles Barthe, François Dupressoir, Benjamin Grégoire, César Kunz, Benedikt Schmidt, and Pierre-Yves Strub. Easycrypt: A tutorial. In Alessandro Aldini,

- Javier López, and Fabio Martinelli, editors, *Foundations of Security Analysis and Design VII - FOSAD 2012/2013 Tutorial Lectures*, volume 8604 of *Lecture Notes in Computer Science*, pages 146–166. Springer, 2013.
9. Gilles Barthe, Benjamin Grégoire, Sylvain Heraud, and Santiago Zanella Béguelin. Computer-aided security proofs for the working cryptographer. In Phillip Rogaway, editor, *CRYPTO 2011*, volume 6841 of *LNCS*, pages 71–90. Springer, Berlin, Heidelberg, August 2011.
 10. Mihir Bellare, Ruth Ng, and Björn Tackmann. Nonces are noticed: AEAD revisited. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part I*, volume 11692 of *LNCS*, pages 235–265. Springer, Cham, August 2019.
 11. Mihir Bellare and Phillip Rogaway. The security of triple encryption and a framework for code-based game-playing proofs. In Serge Vaudenay, editor, *EUROCRYPT 2006*, volume 4004 of *LNCS*, pages 409–426. Springer, Berlin, Heidelberg, May / June 2006.
 12. Daniel J. Bernstein. Curve25519: New Diffie-Hellman speed records. In Moti Yung, Yevgeniy Dodis, Aggelos Kiayias, and Tal Malkin, editors, *PKC 2006*, volume 3958 of *LNCS*, pages 207–228. Springer, Berlin, Heidelberg, April 2006.
 13. Karthikeyan Bhargavan, Cédric Fournet, Markulf Kohlweiss, Alfredo Pironti, and Pierre-Yves Strub. Implementing TLS with verified cryptographic security. In *2013 IEEE Symposium on Security and Privacy*, pages 445–459. IEEE Computer Society Press, May 2013.
 14. Bruno Blanchet. A computationally sound mechanized prover for security protocols. In *2006 IEEE Symposium on Security and Privacy*, pages 140–154. IEEE Computer Society Press, May 2006.
 15. Chris Brzuska, Eric Cornelissen, and Konrad Kohbrok. Security analysis of the MLS key derivation. In *2022 IEEE Symposium on Security and Privacy*, pages 2535–2553. IEEE Computer Society Press, May 2022.
 16. Chris Brzuska, Antoine Delignat-Lavaud, Christoph Egger, Cédric Fournet, Konrad Kohbrok, and Markulf Kohlweiss. Key-schedule security for the TLS 1.3 standard. In Shweta Agrawal and Dongdai Lin, editors, *ASIACRYPT 2022, Part I*, volume 13791 of *LNCS*, pages 621–650. Springer, Cham, December 2022.
 17. Chris Brzuska, Antoine Delignat-Lavaud, Cédric Fournet, Konrad Kohbrok, and Markulf Kohlweiss. State separation for code-based game-playing proofs. In Thomas Peyrin and Steven Galbraith, editors, *ASIACRYPT 2018, Part III*, volume 11274 of *LNCS*, pages 222–249. Springer, Cham, December 2018.
 18. Chris Brzuska and Sabine Oechsner. A state-separating proof for yao’s garbling scheme. In *CSF 2023 Computer Security Foundations Symposium*, pages 137–152. IEEE Computer Society Press, July 2023.
 19. Ran Canetti, Alley Stoughton, and Mayank Varia. EasyUC: Using EasyCrypt to mechanize proofs of universally composable security. In Stephanie Delaune and Limin Jia, editors, *CSF 2019 Computer Security Foundations Symposium*, pages 167–183. IEEE Computer Society Press, 2019.
 20. Whitfield Diffie and Martin E. Hellman. New directions in cryptography. *IEEE Transactions on Information Theory*, 22(6):644–654, 1976.
 21. Danny Dolev and Andrew Chi-Chih Yao. On the security of public key protocols. *IEEE Trans. Inf. Theory*, 29(2):198–207, 1983.
 22. François Dupressoir, Konrad Kohbrok, and Sabine Oechsner. Bringing state-separating proofs to EasyCrypt A security proof for cryptobox. In *CSF 2022 Computer Security Foundations Symposium*, pages 227–242. IEEE Computer Society Press, August 2022.

23. François Dupressoir, Andreas Hülsing, Cameron Low, Matthias Meijers, Charlotte Mylog, and Sabine Oechsner. How hard can it be to formalize a proof? lessons from formalizing CryptoBox three times in EasyCrypt. Cryptology ePrint Archive, Paper 2025/1569, 2025.
24. Eduarda S.V. Freire, Dennis Hofheinz, Eike Kiltz, and Kenneth G. Paterson. Non-interactive key exchange. Cryptology ePrint Archive, Report 2012/732, 2012.
25. Victor Shoup. Sequences of games: a tool for taming complexity in security proofs. Cryptology ePrint Archive, Report 2004/332, 2004.

A Appendix

B Quantitative Metrics

For full transparency, we include some typical quantitative metrics here. While these numbers seem to paint a clear picture, upon further inspection, they actually yield little insight.

- Time
 - Bellare-Rogaway: ~ 2 weeks
 - SSP: ~ 6 months
 - Blended: ~ 6 months

Note that for the SSP and blended approaches, the time also includes understanding how to express this proof style in EasyCrypt. The game-playing approach on the other hand could benefit from years of experience in writing EasyCrypt proofs in this style. Hence these numbers do not reflect a fair comparison.

- Lines of code
 - Bellare-Rogaway: ~ 1500
 - SSP: ~ 2400
 - Blended: ~ 1200 for stepwise SSP, and ~ 1500 for (original) blended
- Lines of code are also misleading as a metric since that would also include overhead such as definitions.

When starting the study, we thought that invariant size might be a good proxy measure for proof complexity. However, we found that the complexity of a proof seems to be more closely related to the *nature* of the invariants than to their *size*, leading again to a qualitative comparison.

C Figures

$\mathcal{O}_{N,E}^{\text{InterNBPKAE}}$	
vars $\mathcal{R}, [\bar{\mathcal{K}}\bar{\mathcal{M}}], \log_n, \log_e$ Init() $\mathcal{R} \leftarrow []$ $[\bar{\mathcal{K}}\bar{\mathcal{M}}] \leftarrow []$ $\log_n \leftarrow \{\}$ $\log_e \leftarrow \{\}$	
Enc (pks, pkr, n, p)	Dec (pkr, pks, n, c)
$\text{sid} \leftarrow \text{gensid}(\text{pks}, \text{pkr})$ if $\mathcal{R}[\text{pks}] \neq \diamond \wedge \text{pkr} \in \mathcal{R}$ $\quad \wedge \text{sid} \notin \log_n :$ if $\mathcal{R}[\text{pkr}] = \diamond :$ $\quad k \leftarrow \text{N.SharedKey}(\mathcal{R}[\text{pks}], \text{pkr})$ else : if $\text{sid} \notin \bar{\mathcal{K}}\bar{\mathcal{M}} :$ $\quad \bar{\mathcal{K}}\bar{\mathcal{M}}[\text{sid}] \leftarrow \\$\mathcal{U}(\bar{\mathcal{K}})$ $\quad k \leftarrow \bar{\mathcal{K}}\bar{\mathcal{M}}[\text{sid}]$ $c \leftarrow \text{E.Enc}(\mathcal{R}[\text{pks}], \text{pkr}, n, p)$ $\log_n \leftarrow \log_n \cup \{(i, n)\}$ $\log_e \leftarrow \log_e \cup \{(i, n, c)\}$ return c return $\perp$	$\text{sid} \leftarrow \text{gensid}(\text{pks}, \text{pkr})$ if $\mathcal{R}[\text{pkr}] \neq \diamond \wedge \text{pks} \in \mathcal{R}$ $\quad \wedge (\text{sid}, n, c) \notin \log_e :$ if $\mathcal{R}[\text{pks}] = \diamond :$ $\quad k \leftarrow \text{N.SharedKey}(\mathcal{R}[\text{pkr}], \text{pks})$ else : if $\text{sid} \notin \bar{\mathcal{K}}\bar{\mathcal{M}} :$ $\quad \bar{\mathcal{K}}\bar{\mathcal{M}}[\text{sid}] \leftarrow \\$\mathcal{U}(\bar{\mathcal{K}})$ $\quad k \leftarrow \bar{\mathcal{K}}\bar{\mathcal{M}}[\text{sid}]$ return $\text{E.Dec}(\mathcal{R}[\text{pkr}], \text{pks}, n, c)$ return $\perp$

Fig. 19. Encryption and decryption oracles considered in the intermediate game. The dashed outline represents changes from the $\mathcal{O}^{\text{RealNBPKAE}}$ oracle.

O_0	
$\text{Enc}(\text{pks}, \text{pkr}, n, p)$	$\text{Dec}(\text{pkr}, \text{pks}, n, c)$
$\text{sid} \leftarrow \text{gensid}(\text{pks}, \text{pkr})$ if $\mathcal{R}[\text{pks}] \neq \diamond \wedge \text{pkr} \in \mathcal{R}$ $\wedge \text{sid} \notin \log_n :$ if $\mathcal{R}[\text{pkr}] = \diamond :$ $k \leftarrow O_N.\text{CorruptReveal}(\text{pks}, \text{pkr})$ else : $k \leftarrow O_N.\text{Test}(\text{pks}, \text{pkr})$ $c \leftarrow E.\text{Enc}(k, n, p)$ $\log_n \leftarrow \log_n \cup \{(i, n)\}$ $\log_e \leftarrow \log_e \cup \{(i, n, c)\}$ return c return $\perp$	$\text{sid} \leftarrow \text{gensid}(\text{pks}, \text{pkr})$ if $\mathcal{R}[\text{pkr}] \neq \diamond \wedge \text{pks} \in \mathcal{R}$ $\wedge (\text{sid}, n, c) \notin \log_e :$ if $\mathcal{R}[\text{pks}] = \diamond :$ $k \leftarrow O_N.\text{CorruptReveal}(\text{pkr}, \text{pks})$ else : $k \leftarrow O_N.\text{Test}(\text{pkr}, \text{pks})$ return $E.\text{Dec}(k, n, c)$ return $\perp$

Fig. 20. Encryption and decryption oracles implemented by $\mathcal{R}_0$, simulating the real/intermediate NBPKE oracles for $\mathcal{A}$. O_N signifies the NIKE oracle provided to $\mathcal{R}_0$ by its game.

O_1	
$\text{Enc}(\text{pks}, \text{pkr}, n, p)$	$\text{Dec}(\text{pkr}, \text{pks}, n, c)$
$\text{sid} \leftarrow \text{gensid}(\text{pks}, \text{pkr})$ if $\mathcal{R}[\text{pks}] \neq \diamond \wedge \text{pkr} \in \mathcal{R}$ $\wedge \text{sid} \notin \log_n :$ if $\mathcal{R}[\text{pkr}] = \diamond :$ $k \leftarrow N.\text{SharedKey}(\mathcal{R}[\text{pks}], \text{pkr})$ $c \leftarrow E.\text{Enc}(k, n, p)$ else : $O_E.\text{RegisterHonest}(\text{sid})$ $c \leftarrow O_E.\text{Enc}(\text{sid}, n, p)$ $\log_n \leftarrow \log_n \cup \{(i, n)\}$ $\log_e \leftarrow \log_e \cup \{(i, n, c)\}$ return c return $\perp$	$\text{sid} \leftarrow \text{gensid}(\text{pks}, \text{pkr})$ if $\mathcal{R}[\text{pkr}] \neq \diamond \wedge \text{pks} \in \mathcal{R}$ $\wedge (\text{sid}, n, c) \notin \log_e :$ if $\mathcal{R}[\text{pks}] = \diamond :$ $k \leftarrow N.\text{SharedKey}(\mathcal{R}[\text{pkr}], \text{pks})$ $p \leftarrow E.\text{Dec}(k, n, c)$ else : $O_E.\text{RegisterHonest}(\text{sid})$ $p \leftarrow O_E.\text{Dec}(\text{sid}, n, c)$ return p return $\perp$

Fig. 21. Encryption and decryption oracles implemented by $\mathcal{R}_1$, simulating the intermediate/ideal NBPKE oracles for $\mathcal{A}$. O_E signifies the NBPAE oracle provided to $\mathcal{R}_1$ by its game.

O_2	
$\text{KeyGen}(h, n, p)$	$\text{Enc}(h, n, p)$
if $h \notin \mathcal{H}$: $\mathcal{H}[h] \leftarrow \text{size}(\mathcal{H})$ return true	
if $h \in \mathcal{H}$: return $O_{\text{AE}}.\text{Enc}(\mathcal{H}[h], n, p)$ return false	
$\text{Dec}(h, n, c)$	
if $h \in \mathcal{H}$: return $O_{\text{AE}}.\text{Dec}(\mathcal{H}[h], n, c)$ return $\perp$	

Fig. 22. Registration, encryption and decryption oracles implemented by $\mathcal{R}_2$, simulating the real/ideal NBAEs oracles for $\mathcal{B}$. O_{AE} signifies the NBAEprot oracle provided to $\mathcal{R}_2$ by its game.

AE_E^b	
vars $\mathcal{M}$	
$\text{Enc}(h, m, n)$	$\text{Dec}(h, c, n)$
$k \leftarrow \text{Get}(h)$ $\text{hon}_k \leftarrow \text{Hon}(h)$ if $(h, n) \in \mathcal{M}$: return $\perp$ if $b \wedge \text{hon}_k$: $c \leftarrow \$ \text{dk}$ else : $c \leftarrow E.\text{Enc}(k, m, n)$ $\mathcal{M}[h, n] \leftarrow (m, c)$ return c	
$k \leftarrow \text{Get}(h)$ $\text{hon}_k \leftarrow \text{Hon}(h)$ $m \leftarrow \perp$ if $b \wedge \text{hon}_k$: $m \leftarrow \text{getmsg}(\mathcal{M}[h, n], c)$ else : $m \leftarrow E.\text{Dec}(k, c, n)$ return m	

Fig. 23. Oracles of the AE_E^b package.

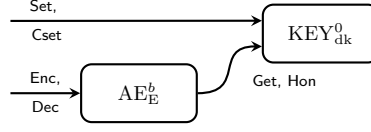


Fig. 24. Package graph for GAE_E^b .

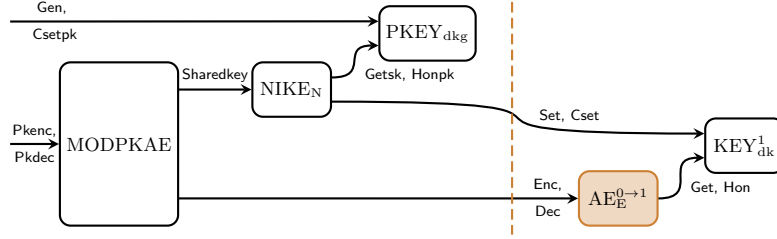


Fig. 25. NBAE reduction: Game GAE_E^b (right of dashed line), reduction $\mathcal{R}_{\text{GAE}}$ (left of dashed line).

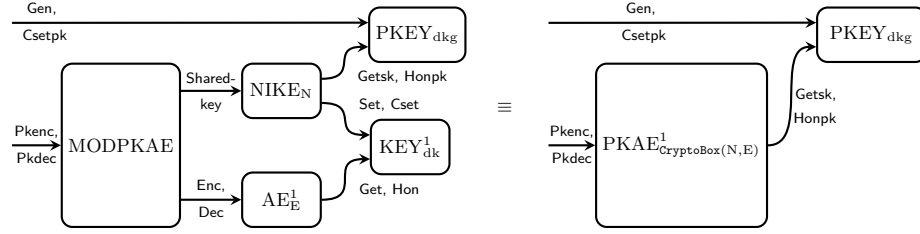


Fig. 26. Reconstruction of modular version into ideal NBPKE game $\text{GPKAE}_{\text{CryptoBox}(N,E)}^1$.

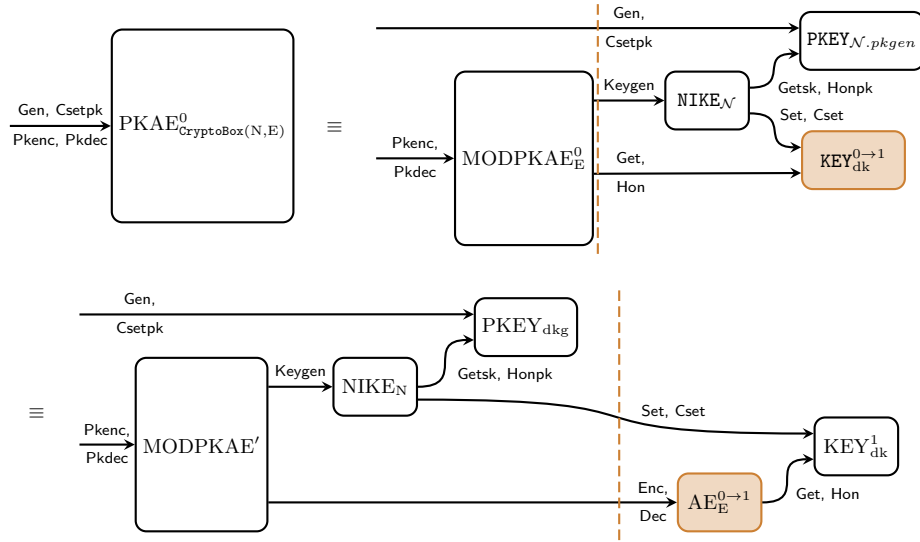


Fig. 27. Stepwise deconstruction of the NBPKE game.

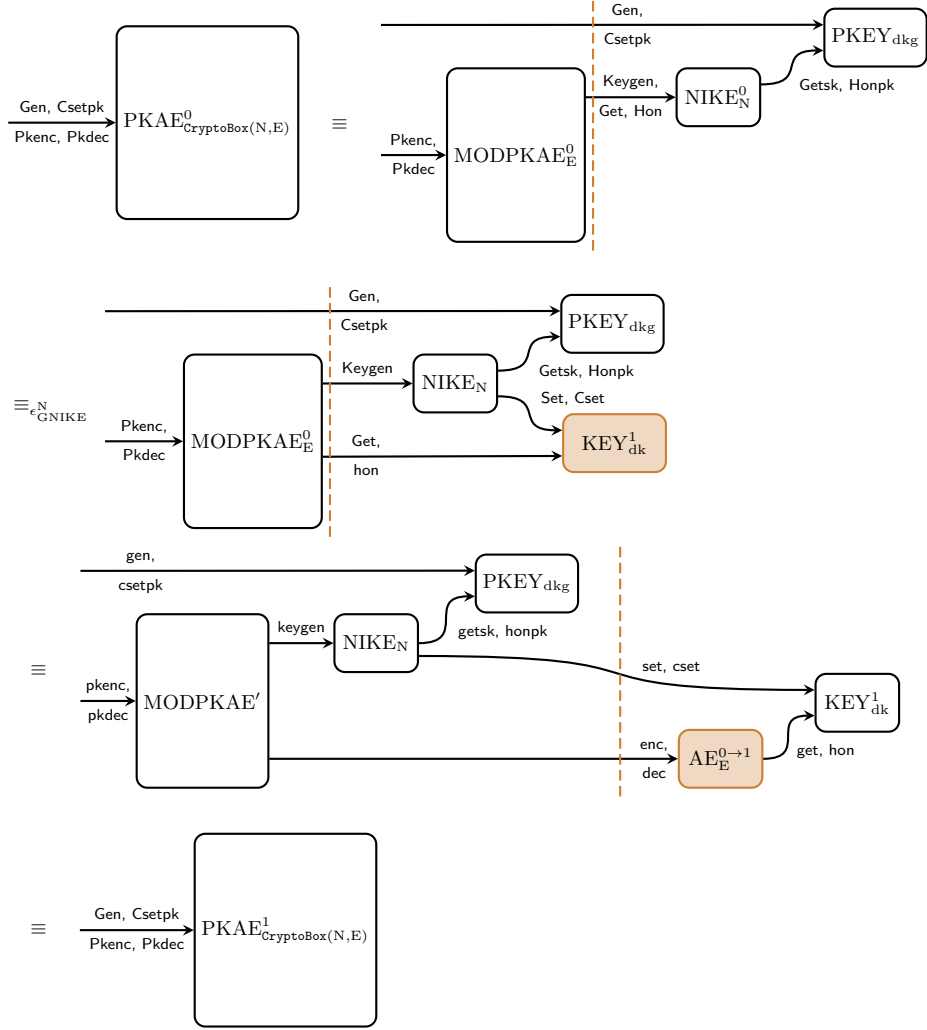


Fig. 28. Proof steps for stepwise approach. Highlighted packages are idealized through the proof and introduce bound on advantage.

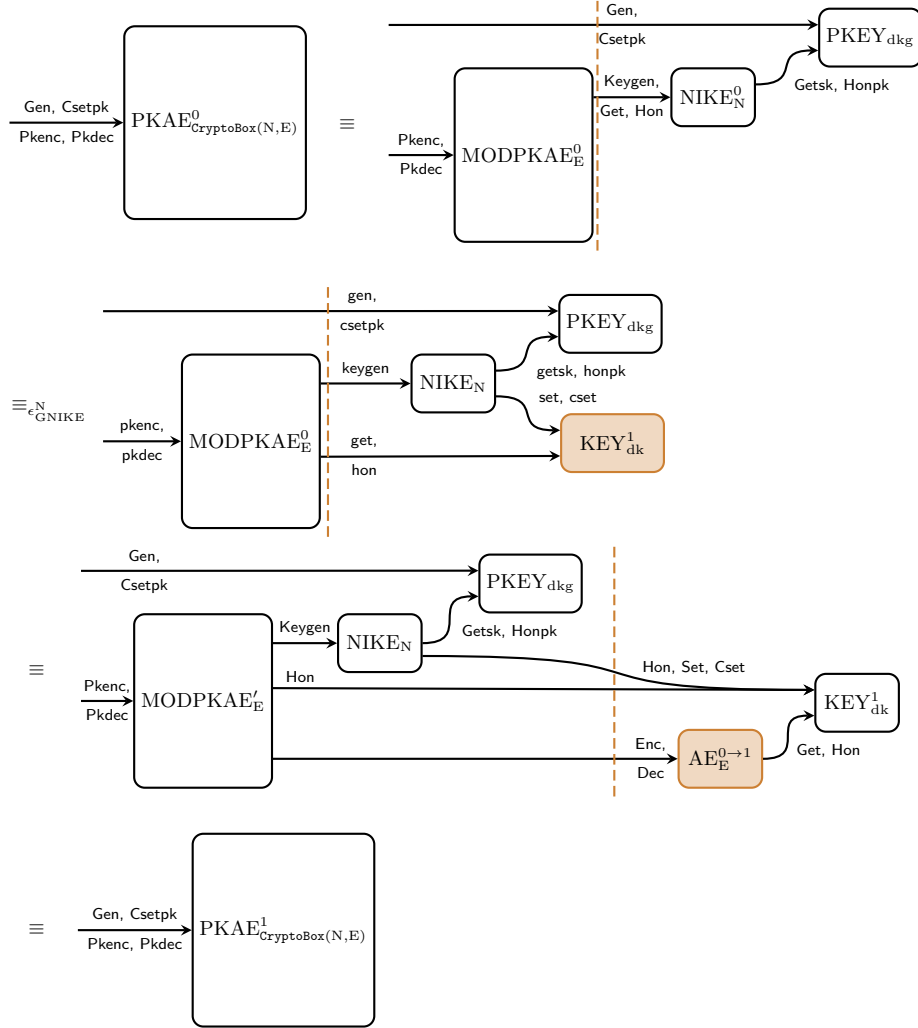


Fig. 29. Proof steps for blended approach. Highlighted packages are idealized through the proof and introduce bound on advantage.